

CS2109S: Introduction to AI and Machine Learning

Lecture 5: Linear Regression

10 February 2026

Chinese New Year Holiday (Next Week)

- No lecture class next week
- Recording of the lecture will be made available
- Some tutorials may be moved online/recorded

Midterm – Reminder

- Date & Time:
 - **Monday, 2nd March 2026**, from **18:30** to 20:**30** (18:15 entry)
 - No additional time will be provided for latecomers.
- Venue:
 - **MPSH 1A & 1B**
- Format:
 - Digital Assessment (**Exemplify**)
- Materials:
 - All topics covered **until and including Lecture 6**
- Cheatsheet:
 - **1 x A4 paper, both sides**
- Calculators:
 - Standard and scientific calculators are allowed.
 - Graphing or programmable calculators are allowed, but please make sure they **do not contain any stored functions or programs.**

More details will be announced later.

Midterm – Exemplify

All the info:

<https://nus.atlassian.net/wiki/spaces/DAstudent/overview>

Video

<https://nus.atlassian.net/wiki/spaces/DAstudent/pages/22511670/Exemplify+Student+Briefing+Video>

Common briefing:

<https://nus.atlassian.net/wiki/spaces/DAstudent/pages/22511675/Common+Briefing+Sessions>

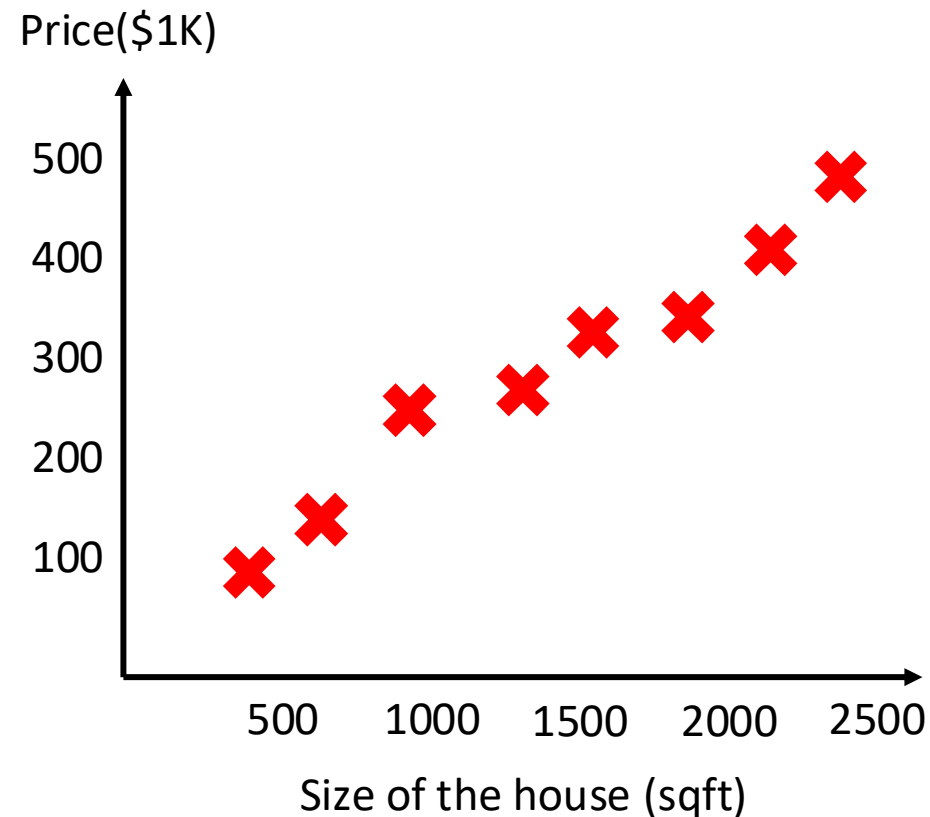
Outline

- **Linear Regression**
 - Data
 - Model
 - Loss
- Learning via Normal Equations
- Learning via Gradient Descent
 - Gradient Descent Algorithm
 - Variants: Mini-batch, stochastic
- Feature Transformation

Example: Housing Price Prediction

Given a house with a certain size (in square feet), what is its price?

Size (sqft)	Price (\$100K)
400	0.9
750	1
950	2.5
1200	2.8
...	...



Data

Suppose:

- We are given n data points.
- Each data point consists of **features** and a **target** variable.
- Each data point has d features and they are described by **real numbers**.
- The target is also a **real number**.

Data – Math

Suppose

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(\textcolor{red}{n})}, y^{(\textcolor{red}{n})})\},$$

where for all $i \in \{1, \dots, \textcolor{red}{n}\}$

Features: $x^{(i)} \in \mathbb{R}^{\textcolor{red}{d}}$

Targets: $y^{(i)} \in \mathbb{R}$

Task

Suppose we are given another data point $x \in \mathbb{R}^d$ and **no** target. Based on the dataset, find a function that predicts the target $y \in \mathbb{R}$ for that x .

This task is called regression.

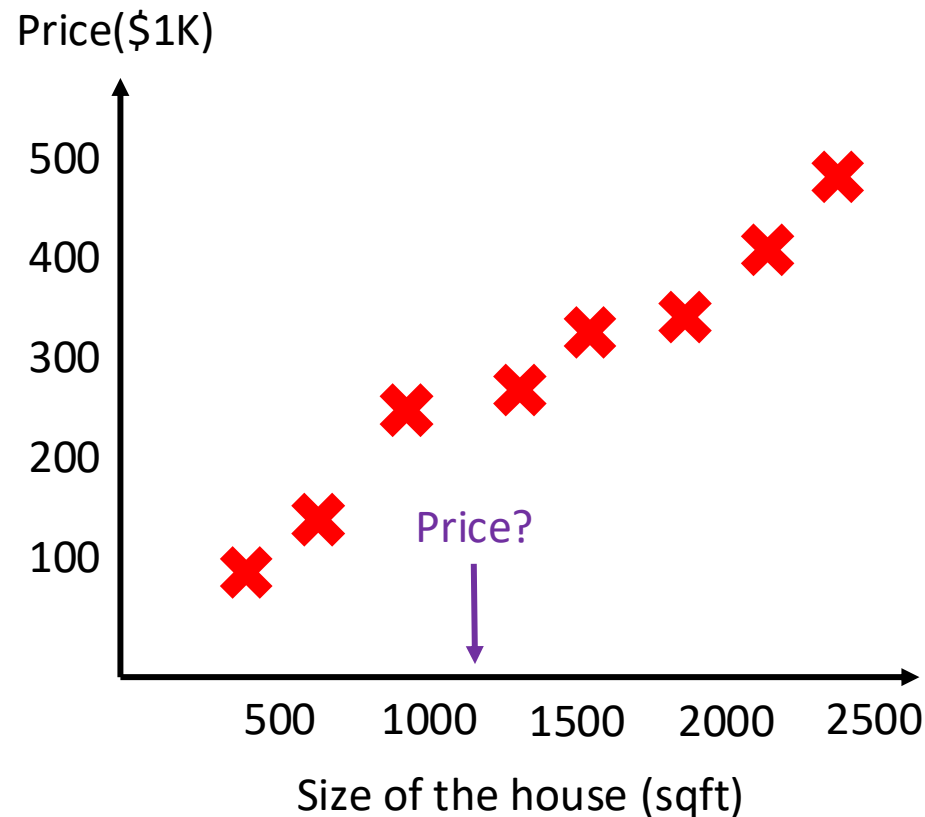
History: the word comes from “to regress”, as in “going back”, reverting to the mean, for example in the context of biology.

Example: Housing Price Prediction

Given a house with a certain size (in square feet), what is its price?

i	$x_1^{(i)}$	$y^{(i)}$
1	400	0.9
2	750	1
3	950	2.5
4	1200	2.8
...	...	...

Dataset D



Example:

x = a house with 1150 sqft

What is the price of x ?

What kind of function could we use?

Linear function $f: \mathbb{R}^d \rightarrow \mathbb{R}$

Background: Vectors and Dot Product

Vectors: Let $w_0, w_1, \dots, w_d$ be real numbers.

Column vector $w = \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_d \end{bmatrix}$

Row vector $w^T = [w_0 \quad w_1 \quad \dots \quad w_d]$

Dot product: Let u, v be two vectors.

$$u^T v = [u_0 \quad u_1 \quad \dots \quad u_d] \begin{bmatrix} v_0 \\ v_1 \\ \dots \\ v_d \end{bmatrix} = \sum_{j=0}^d u_j v_j$$

Linear Model

Given an input vector x of dimension d , a linear model is a function with the following form:

$$h_w(x) = w_0x_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d$$

where $w_0, \dots, w_d$ are **parameters/weights** and $x_0 = 1$ is a dummy variable.

We shorthand this function by using the dot product:

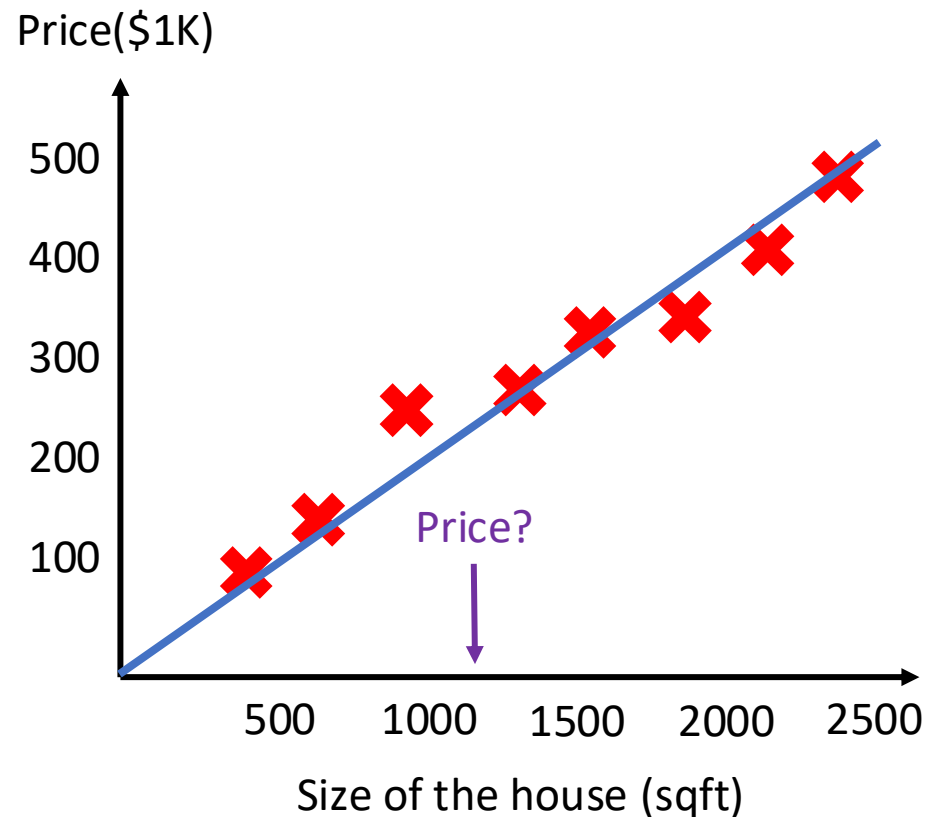
$$h_w(x) = w^T x = \sum_{j=0}^d w_j x_j$$

Example: Housing Price Prediction

Given a house with a certain size (in square feet), what is its price?

i	$x_1^{(i)}$	$y^{(i)}$
1	400	0.9
2	750	1
3	950	2.5
4	1200	2.8
...	...	...

Dataset D



$$h_w(x) = w_0 x_0 + w_1 x_1$$

$x_0 = 1$ (dummy variable)

Linear Model:

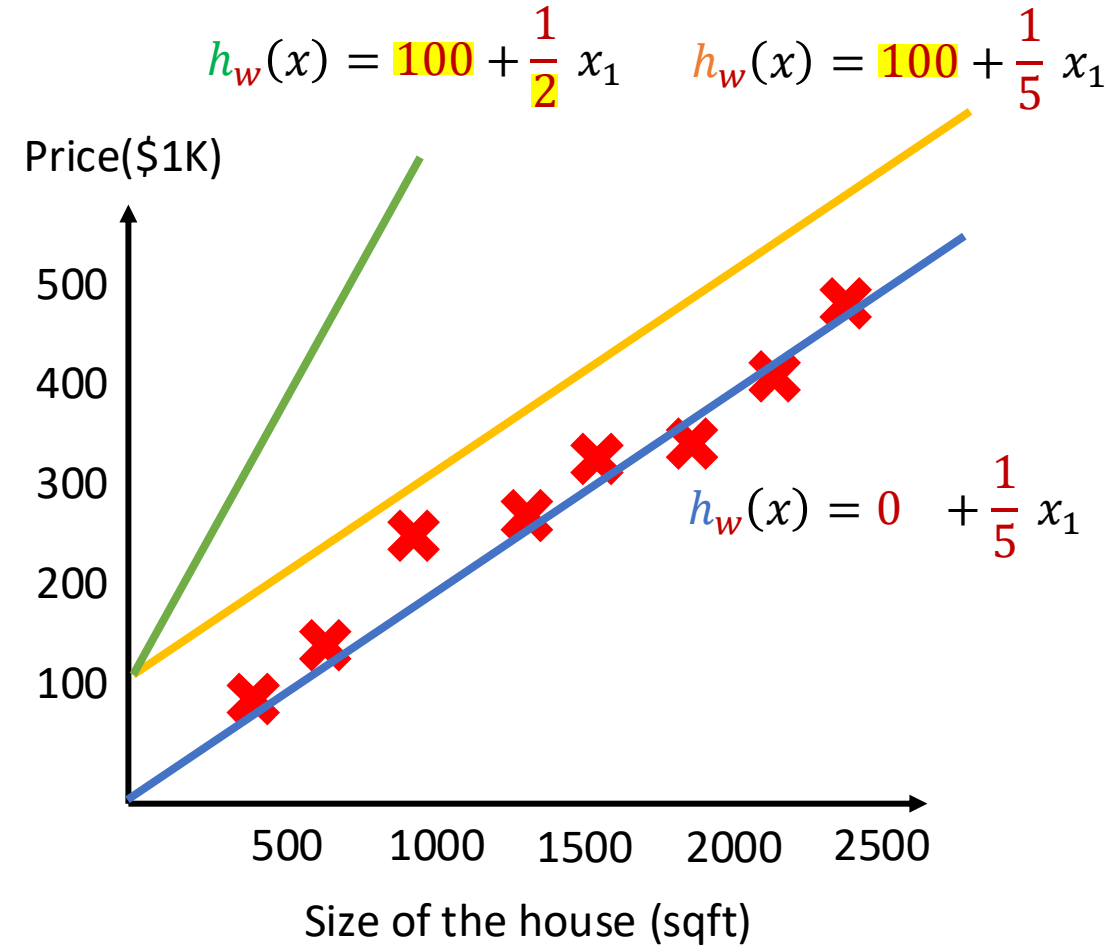
$$h_w(x) = 0 + \frac{1}{5} x_1$$

The Best Linear Model

There are **infinitely many** linear models that could represent the relationship between input x and output.

We want a linear model with the lowest loss on the data.

It is common to use **mean squared error (MSE)** loss for linear regression.



Mean Squared Error (MSE) Loss

Given a dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$ where $y^{(i)} \in \mathbb{R}$,

$$J_{MSE}(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n (h_{\mathbf{w}}(x^{(i)}) - y^{(i)})^2$$

- Linear regression = linear model + MSE loss
- The factor $\frac{1}{2}$ is only for mathematical convenience, i.e., because we take derivatives later.
- Notice that J_{MSE} is a function of $\mathbf{w}$; we want to find $\mathbf{w}$ that minimizes the loss!

Poll Everywhere

Select the correct statements:

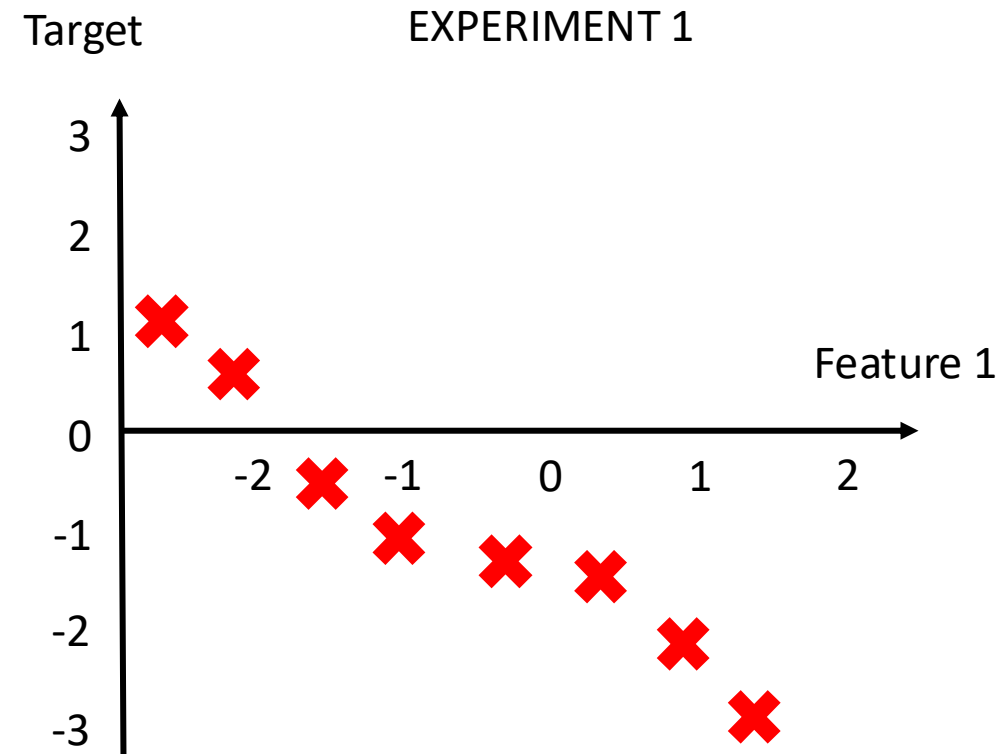
1. Linear regression is used when the **target** is a **categorical** variable.
2. In linear regression, we need to find **features** such that they fit to the weights of the linear model.
3. For EXPERIMENT 1, the best linear model will have $w_0 > 0$.
4. For EXPERIMENT 1, the best linear model will have $w_1 < 0$.
5. None of the above.



Poll Everywhere

Select the correct statements:

1. Linear regression is used when the **target** is a **categorical** variable.
2. In linear regression, we need to find **features** such that they fit to the weights of the linear model.
3. For EXPERIMENT 1, the best linear model will have $w_0 > 0$.
4. For EXPERIMENT 1, the best linear model will have $w_1 < 0$.
5. None of the above.



Outline

- Linear Regression
 - Data
 - Model
 - Loss
- **Learning via Normal Equations**
- Learning via Gradient Descent
 - Gradient Descent Algorithm
 - Variants: Mini-batch, stochastic
- Feature Transformation

The following needs a few:

Basic concepts

- Gradient and partial derivative
- Minimizing a function
- Matrices
- Matrix multiplication

Let's dive in!

Background: Gradient and Partial Derivative

Suppose that you are given a function h as follows.

$$h(\mathbf{w}) = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$

The **gradient** of the function is

$$\nabla h(\mathbf{w}) = \left[\frac{\partial h(\mathbf{w})}{\partial w_1} \quad \frac{\partial h(\mathbf{w})}{\partial w_2} \quad \cdots \quad \frac{\partial h(\mathbf{w})}{\partial w_d} \right]^T = [x_1 \quad x_2 \quad \cdots \quad x_d]^T$$

Here, $\frac{\partial h(\mathbf{w})}{\partial w_i} = x_i$ is called the **partial derivative** of $h(\mathbf{w})$ with respect to w_i .

Note: Gradients are defined for all **differentiable** scalar functions, not just linear functions.

Background: Minimizing a Function

Suppose that you have a 1-dimensional scalar function of variable w , with scalars x and y .

$$f(w) = \frac{1}{2}(wx - y)^2$$

The **minimum** of this function is when the **gradient** $\nabla f(w) = 0$.

$$\nabla f(w) = (wx - y)x$$

$$0 = (wx - y)x$$

$$w = \frac{y}{x}$$

For a function with d -dimensional variable $w = [w_1 \dots w_d]^T$, the **minimum** of the function satisfies that the **partial derivative** $\frac{\partial f(w)}{\partial w_i} = 0$ for all w_i .

Background: Matrices

Let $x_{11}, x_{12}, \dots, x_{1d}, x_{21}, \dots, x_{2d}, \dots, x_{nd}$ be real numbers.

Matrix

$$X = \begin{bmatrix} x_{11} & \dots & x_{1d} \\ \dots & \dots & \dots \\ x_{n1} & \dots & x_{nd} \end{bmatrix} \in \mathbb{R}^{n \times d}$$

Transpose of Matrix

$$X^T = \begin{bmatrix} x_{11} & \dots & x_{n1} \\ \dots & \dots & \dots \\ x_{1d} & \dots & x_{nd} \end{bmatrix} \in \mathbb{R}^{d \times n}$$

Background: Matrices

Matrix-vector multiplication: Let X be a matrix and v be a vector.

$$Xv = \begin{bmatrix} x_{11} & \dots & x_{1d} \\ \dots & \dots & \dots \\ x_{n1} & \dots & x_{nd} \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ \dots \\ v_d \end{bmatrix} = \begin{bmatrix} \sum_{j=1}^d x_{1j} v_j \\ \sum_{j=1}^d x_{2j} v_j \\ \dots \\ \sum_{j=1}^d x_{nj} v_j \end{bmatrix} \in \mathbb{R}^n$$

Matrix multiplication: Let X, A be two matrices (with suitable dimension).

$$XA = \begin{bmatrix} x_{11} & \dots & x_{1d} \\ \dots & \dots & \dots \\ x_{n1} & \dots & x_{nd} \end{bmatrix} \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{d1} & \dots & a_{dn} \end{bmatrix}$$

$$= \begin{bmatrix} \sum_{j=1}^d x_{1j} a_{j1} & \dots & \sum_{j=1}^d x_{1j} a_{jn} \\ \dots & \dots & \dots \\ \sum_{j=1}^d x_{nj} a_{j1} & \dots & \sum_{j=1}^d x_{nj} a_{jn} \end{bmatrix} \in \mathbb{R}^{n \times n}$$

Data – Matrix/Vector form

We view the **data input** for linear regression as a matrix and a vector.

Data points of features:

$$X = \begin{bmatrix} 1 & x_1^{(1)} & & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ 1 & \vdots & & \vdots \\ 1 & x_1^{(n)} & & x_d^{(n)} \end{bmatrix}$$

This matrix is often called **data matrix** or **design matrix**.

Targets:

$$Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(n)} \end{bmatrix}$$

Finding the Best Linear Regression Model

The best linear regression model is the one that has the **minimum** loss.

$$J_{MSE}(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n (h_{\mathbf{w}}(x^{(i)}) - y^{(i)})^2$$

The **minimum** of this function satisfies:

- Gradient $\nabla J_{MSE}(\mathbf{w}) = 0$.
- In other words: Partial derivative $\frac{\partial J_{MSE}(\mathbf{w})}{\partial w_i} = 0$ for all w_i .

$$\begin{aligned} & \frac{\partial J_{MSE}(\mathbf{w})}{\partial w_j} \\ &= \frac{1}{2n} \frac{\partial}{\partial w_j} \sum_{i=1}^n (h_{\mathbf{w}}(x^{(i)}) - y^{(i)})^2 \\ &= \frac{1}{n} \sum_{i=1}^n (h_{\mathbf{w}}(x^{(i)}) - y^{(i)}) \frac{\partial}{\partial w_j} h_{\mathbf{w}}(x^{(i)}) \\ &= \frac{1}{n} \sum_{i=1}^n (h_{\mathbf{w}}(x^{(i)}) - y^{(i)}) x_j^{(i)} \end{aligned}$$

Normal Equation

The best linear regression model is the one that has the **minimum** loss, which is when the **partial derivative**

$$\frac{\partial J_{MSE}(\mathbf{w})}{\partial w_i} = 0$$

for all w_i .

Find the best $\mathbf{w}$ by solving a closed-form expression:

$$\mathbf{w} = (X^T X)^{-1} X^T Y$$

$$\frac{\partial J_{MSE}(\mathbf{w})}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)} = 0$$

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix}$$

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_d \end{bmatrix}$$

$$Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(n)} \end{bmatrix}$$

Express this with
vectors and matrices

$$X^T (X\mathbf{w} - Y) = 0$$

Rearrange

$$X^T X\mathbf{w} = X^T Y$$

Assume **invertible**

$$\frac{\partial J_{MSE}(\mathbf{w})}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)} = 0$$

$$\begin{bmatrix} \frac{\partial J_{MSE}(\mathbf{w})}{\partial w_0} \\ \frac{\partial J_{MSE}(\mathbf{w})}{\partial w_1} \\ \vdots \\ \frac{\partial J_{MSE}(\mathbf{w})}{\partial w_d} \end{bmatrix} = \frac{1}{n} \begin{bmatrix} \left(w_0 + w_1 x_1^{(1)} + \dots + w_d x_d^{(1)} - y^{(1)} \right) 1 + \dots + \left(w_0 + w_1 x_1^{(n)} + \dots + w_d x_d^{(n)} - y^{(n)} \right) 1 \\ \left(w_0 + w_1 x_1^{(1)} + \dots + w_d x_d^{(1)} - y^{(1)} \right) x_1^{(1)} + \dots + \left(w_0 + w_1 x_1^{(n)} + \dots + w_d x_d^{(n)} - y^{(n)} \right) x_1^{(n)} \\ \vdots \\ \left(w_0 + w_1 x_1^{(1)} + \dots + w_d x_d^{(1)} - y^{(1)} \right) x_d^{(1)} + \dots + \left(w_0 + w_1 x_1^{(n)} + \dots + w_d x_d^{(n)} - y^{(n)} \right) x_d^{(n)} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(n)} \\ \vdots & \vdots & \dots & \vdots \\ x_d^{(1)} & x_d^{(2)} & \dots & x_d^{(n)} \end{bmatrix} \begin{bmatrix} w_0 + w_1 x_1^{(1)} + \dots + w_d x_d^{(1)} - y^{(1)} \\ w_0 + w_1 x_1^{(2)} + \dots + w_d x_d^{(2)} - y^{(2)} \\ \vdots \\ w_0 + w_1 x_1^{(n)} + \dots + w_d x_d^{(n)} - y^{(n)} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(n)} \\ \vdots & \vdots & \dots & \vdots \\ x_d^{(1)} & x_d^{(2)} & \dots & x_d^{(n)} \end{bmatrix} \left(\begin{bmatrix} w_0 + w_1 x_1^{(1)} + \dots + w_d x_d^{(1)} \\ w_0 + w_1 x_1^{(2)} + \dots + w_d x_d^{(2)} \\ \vdots \\ w_0 + w_1 x_1^{(n)} + \dots + w_d x_d^{(n)} \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(n)} \\ \vdots & \vdots & \dots & \vdots \\ x_d^{(1)} & x_d^{(2)} & \dots & x_d^{(n)} \end{bmatrix} \left(\begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \dots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$X^T (X\mathbf{w} - Y) = 0$$

Express with
vectors and
matrices

The Problems with Normal Equation

For linear regression (linear model + MSE), the normal equation solves the problem of finding the best parameters.

However:

- The cost of normal equation is d^3 (for inverting matrix) .
- It will not work if $(X^T X)^{-1}$ is **not invertible**.
- It will not work for **non-linear models**, which we will introduce in the future lectures.

Is there any alternative method to **find a minimum of a function?**

Break

HOW STATISTICS CAN BE MISLEADING

TEDEd



Outline

- Linear Regression
 - Data
 - Model
 - Loss
- Learning via Normal Equations
- **Learning via Gradient Descent**
 - Gradient Descent Algorithm
 - Variants: Mini-batch, stochastic
- Feature Transformation

Background: Gradient and the Minimum

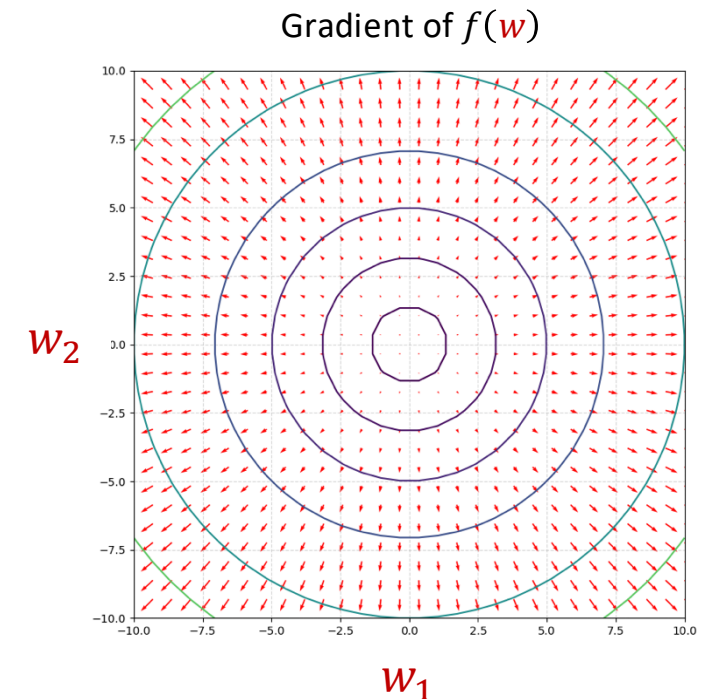
Suppose that you have the following function.

$$f(\mathbf{w}) = (w_1)^2 + (w_2)^2$$

The gradient is

$$\nabla f(\mathbf{w}) = \begin{bmatrix} \frac{\partial f(\mathbf{w})}{\partial w_1} \\ \frac{\partial f(\mathbf{w})}{\partial w_2} \end{bmatrix} = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix}$$

Notice that the gradient vector is pointing away from the minimum of $f(\mathbf{x})$ at $w_1 = w_2 = 0$.



Gradient Descent

- Start at some w (e.g., randomly initialized).
- Update w with a step in the opposite direction of the gradient (i.e., towards lower loss)

$$w_j \leftarrow w_j - \underbrace{\gamma}_{\text{Learning Rate}} \frac{\partial J(w_0, w_1, \dots)}{\partial w_j}.$$

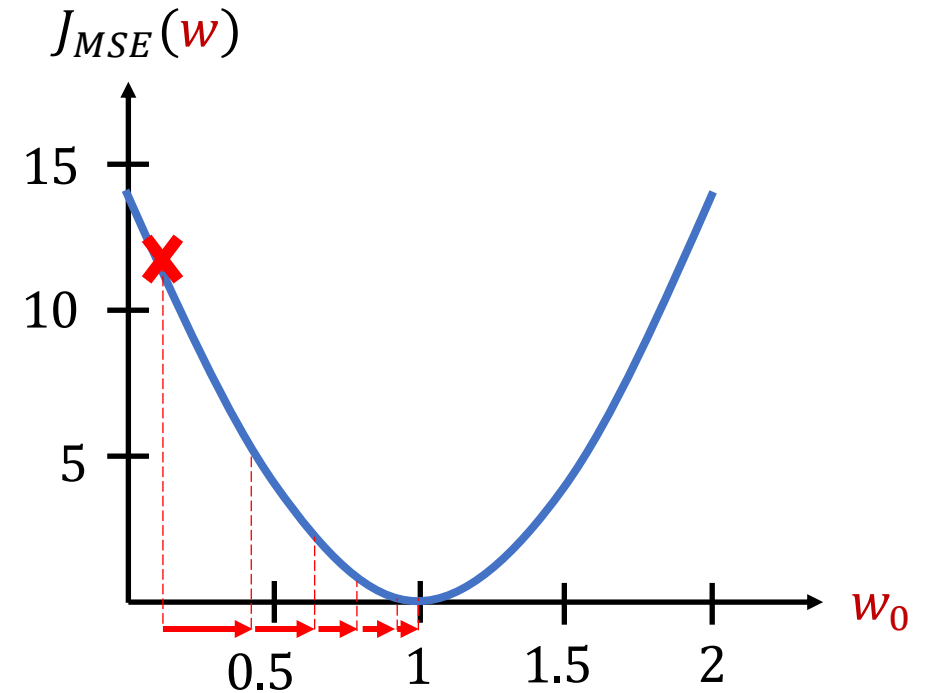
- Learning rate $\gamma > 0$ is a hyperparameter that determines the step size.
- Repeat until termination criterion is satisfied.
 - E.g., change between steps is small, maximum number of steps is reached, etc.

Gradient Descent: Example with 1 Parameter

- Start at some w_0 .
- Update w_0 with

$$w_0 \leftarrow w_0 - \underbrace{\gamma}_{\text{Learning Rate}} \frac{\partial J(w_0)}{\partial w_0}$$

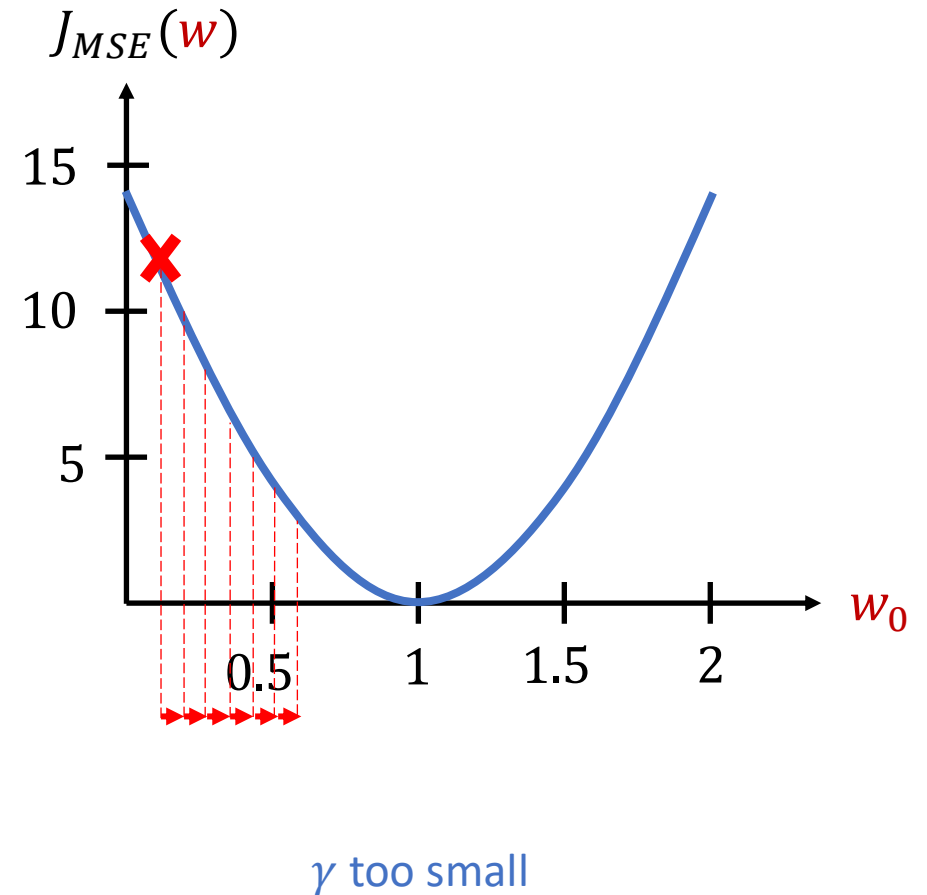
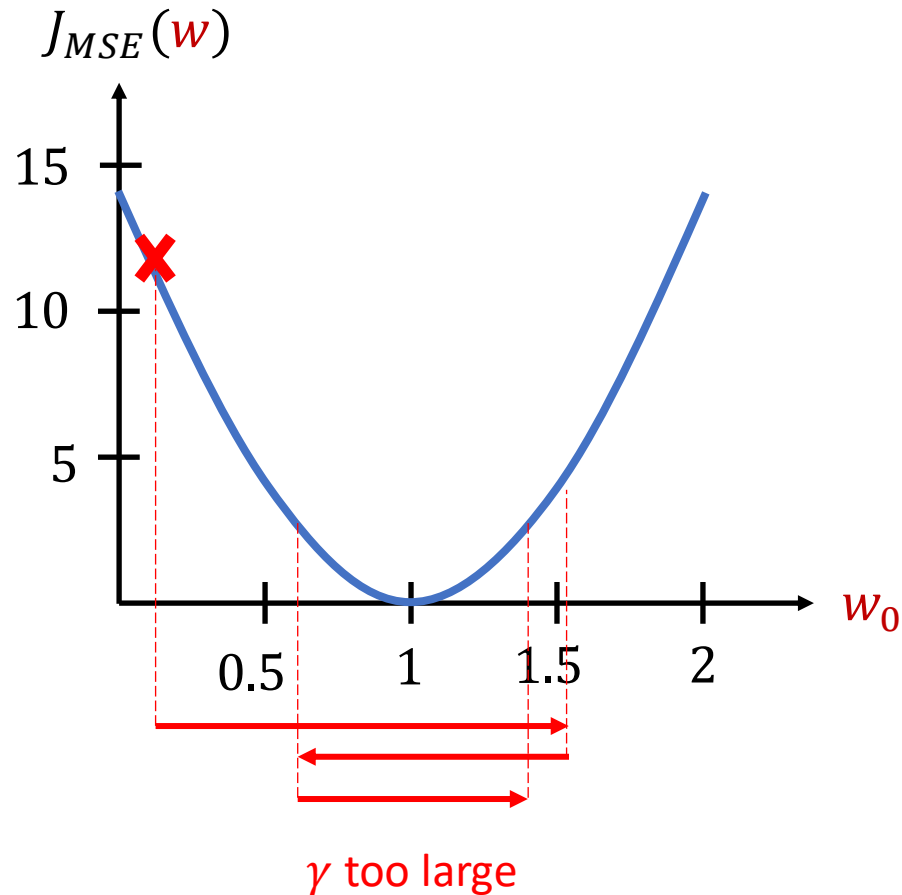
- Repeat until termination criterion is satisfied.



As it gets closer to a minimum,

- The magnitude of the **slope** becomes smaller
- The **step size** become smaller

Gradient Descent: Setting the Learning Rate

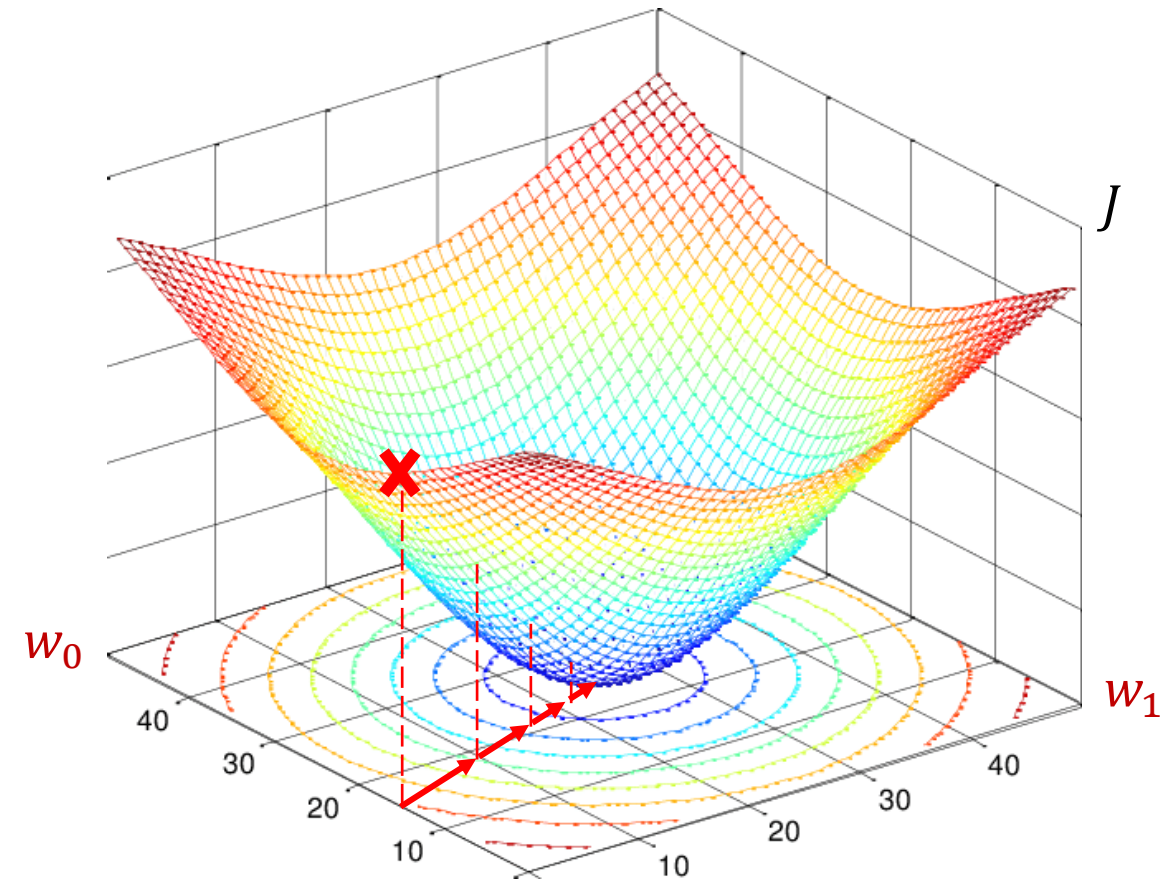


Gradient Descent: Example with 2 Parameters

- Start at some w_0, w_1 .
- Update w_0, w_1 with

$$w_j \leftarrow w_j - \gamma \frac{\partial J(w_0, w_1)}{\partial w_j}$$

- Repeat until termination criterion is satisfied.



Gradient Descent: A Common Mistake

- Start at some w_0, w_1 .
- Update w_0, w_1 with

$$w_j \leftarrow w_j - \gamma \frac{\partial J(w_0, w_1)}{\partial w_j}$$

- Repeat until termination criterion is satisfied.

Which implementation is correct?

Gradient is computed based on the updated w_0

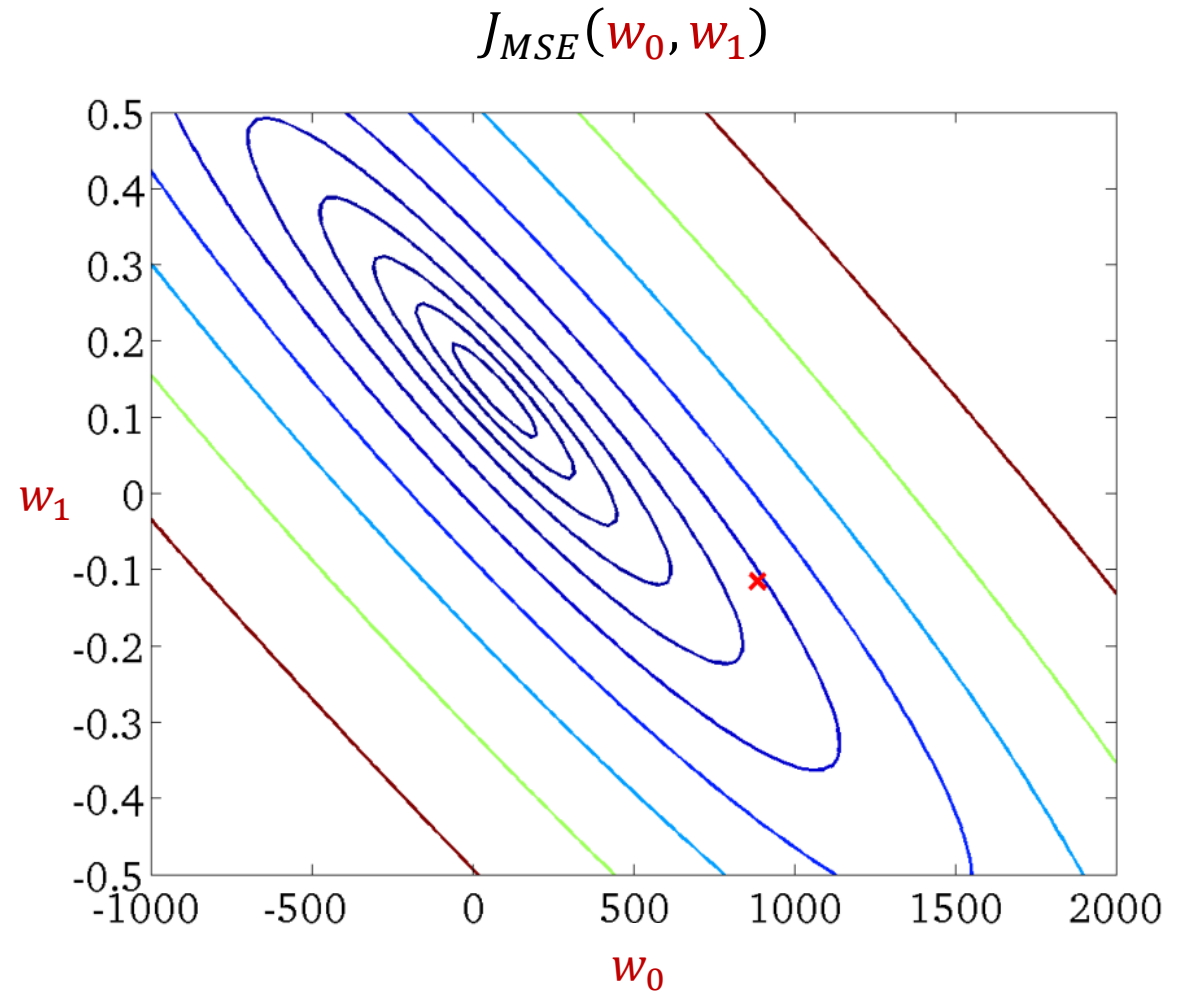
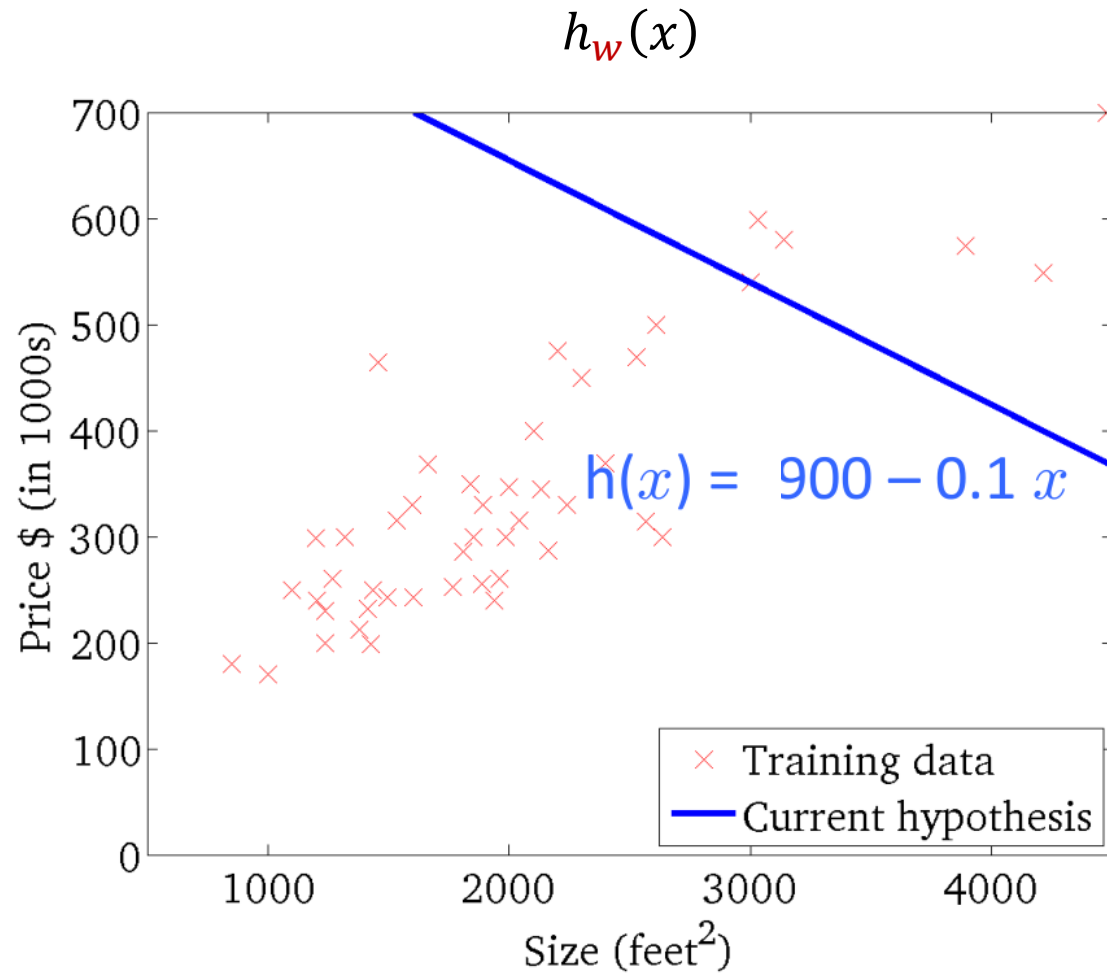
$$\begin{aligned} w_0 &= w_0 - \gamma \frac{\partial J(w_0, w_1)}{\partial w_0} \\ w_1 &= w_1 - \gamma \frac{\partial J(w_0, w_1)}{\partial w_1} \end{aligned}$$



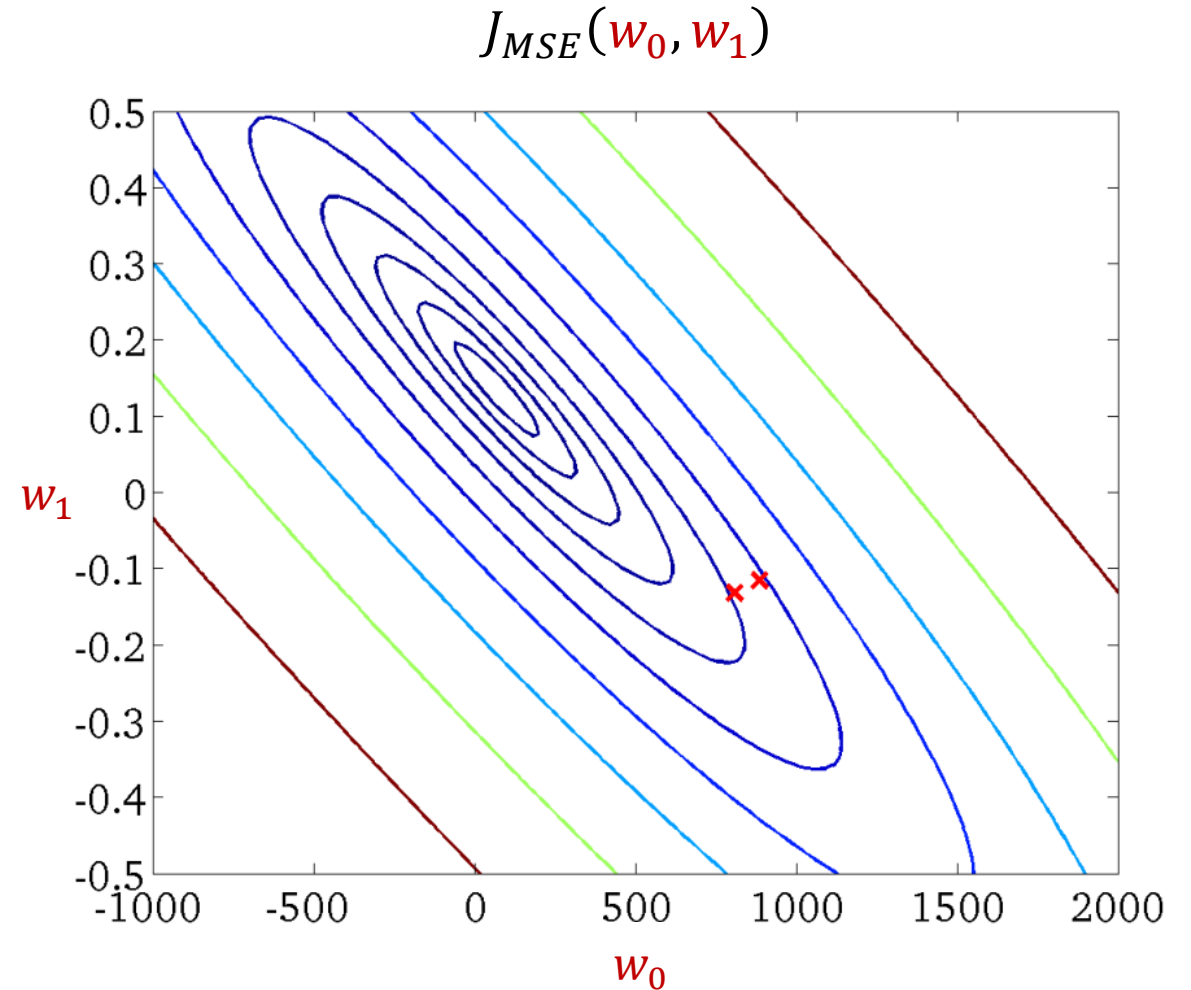
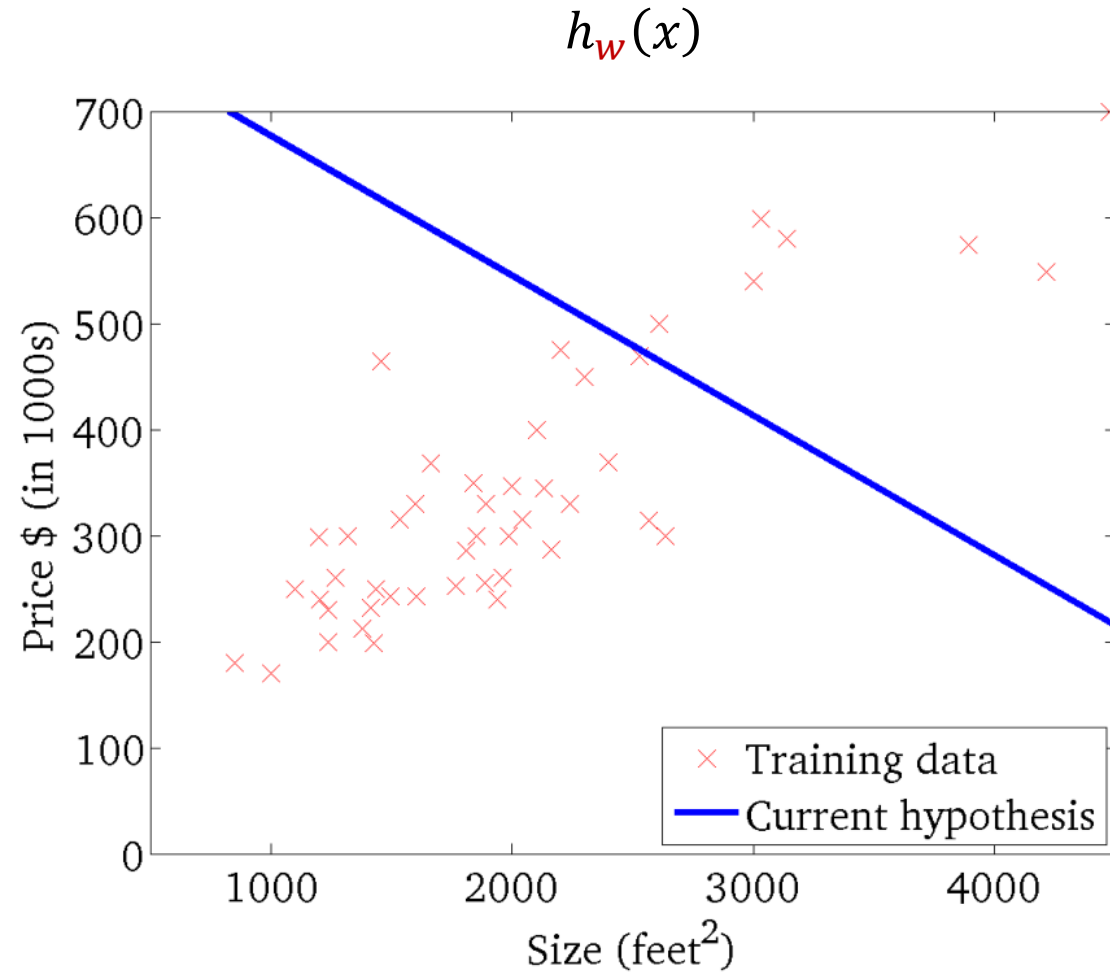
$$\begin{aligned} a &= \frac{\partial J(w_0, w_1)}{\partial w_0} \\ b &= \frac{\partial J(w_0, w_1)}{\partial w_1} \\ w_0 &= w_0 - \gamma a \\ w_1 &= w_1 - \gamma b \end{aligned}$$



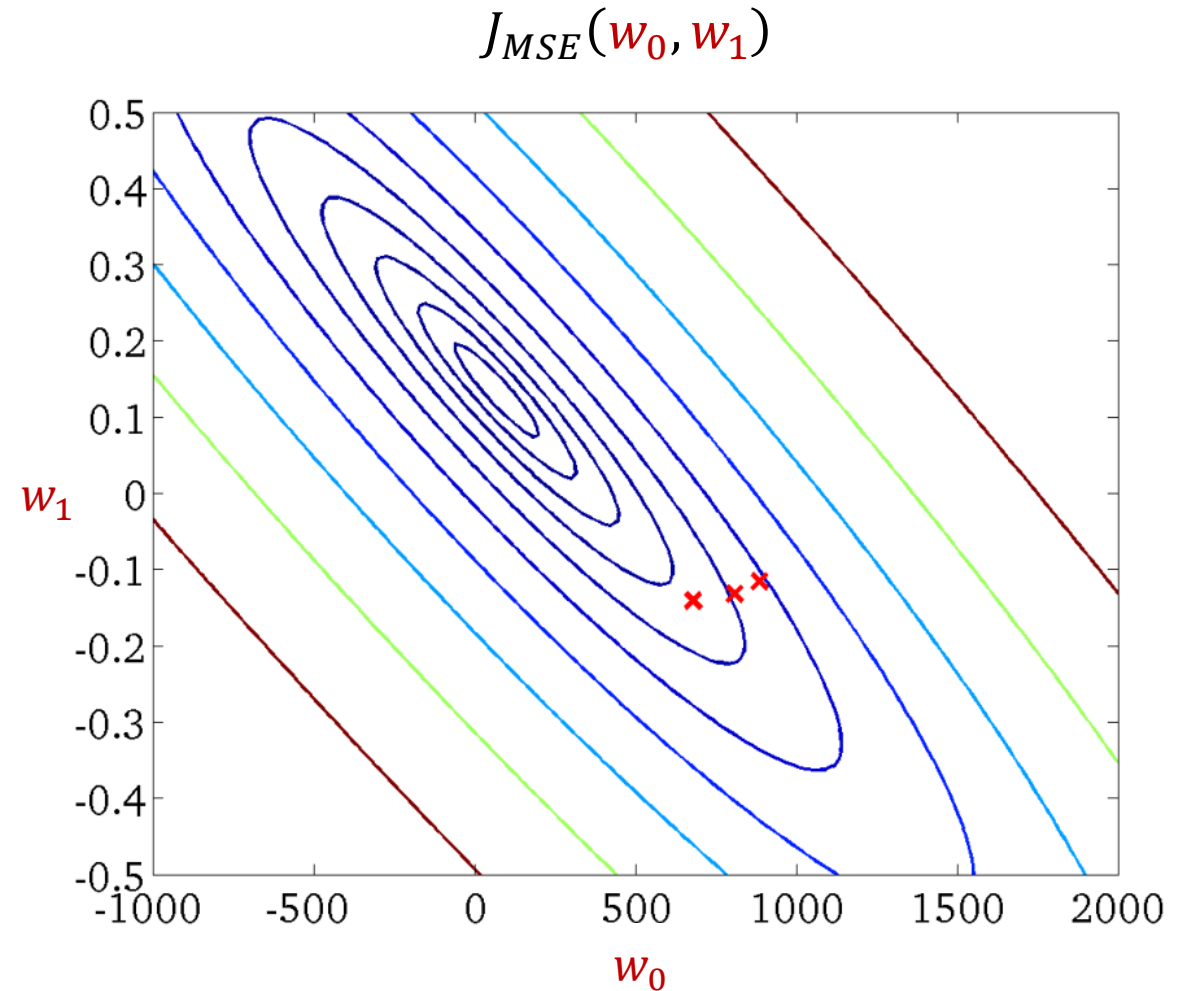
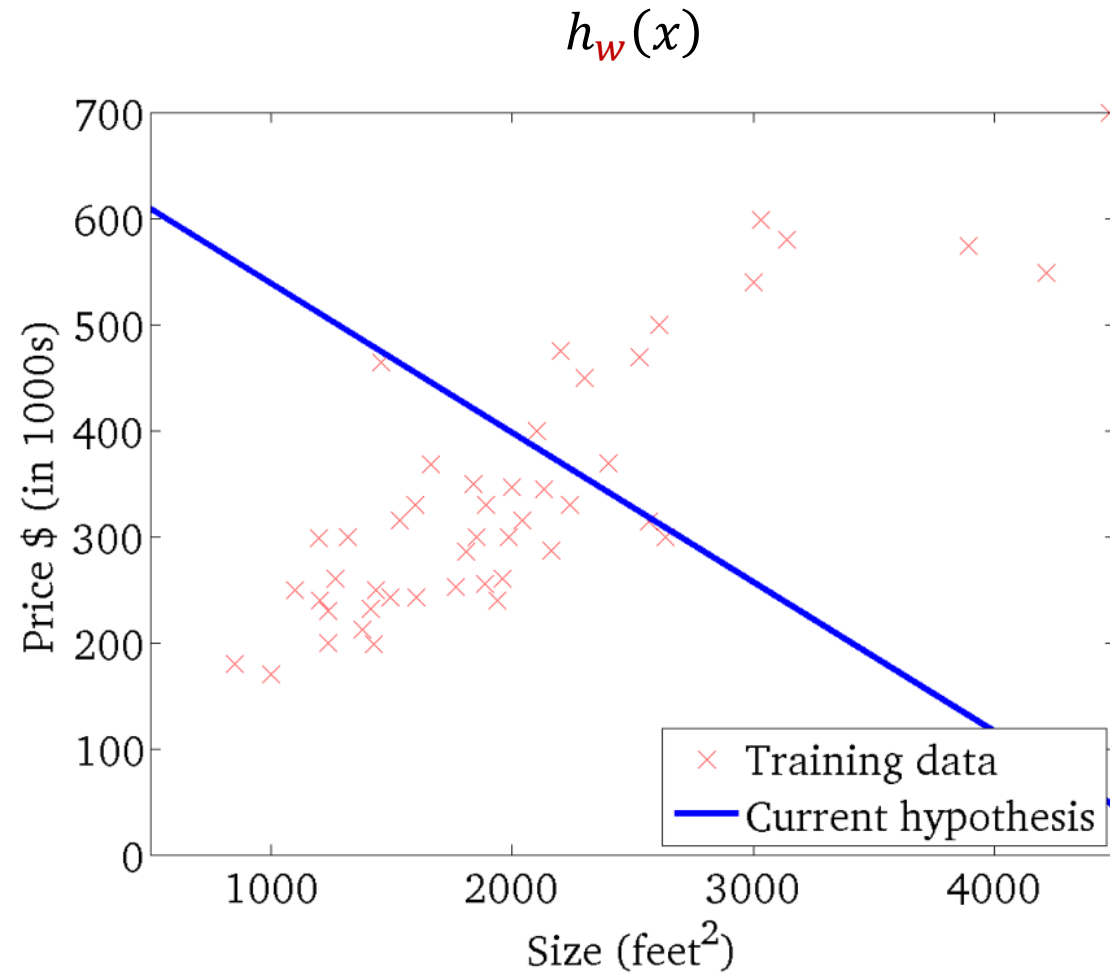
Linear Regression with Gradient Descent



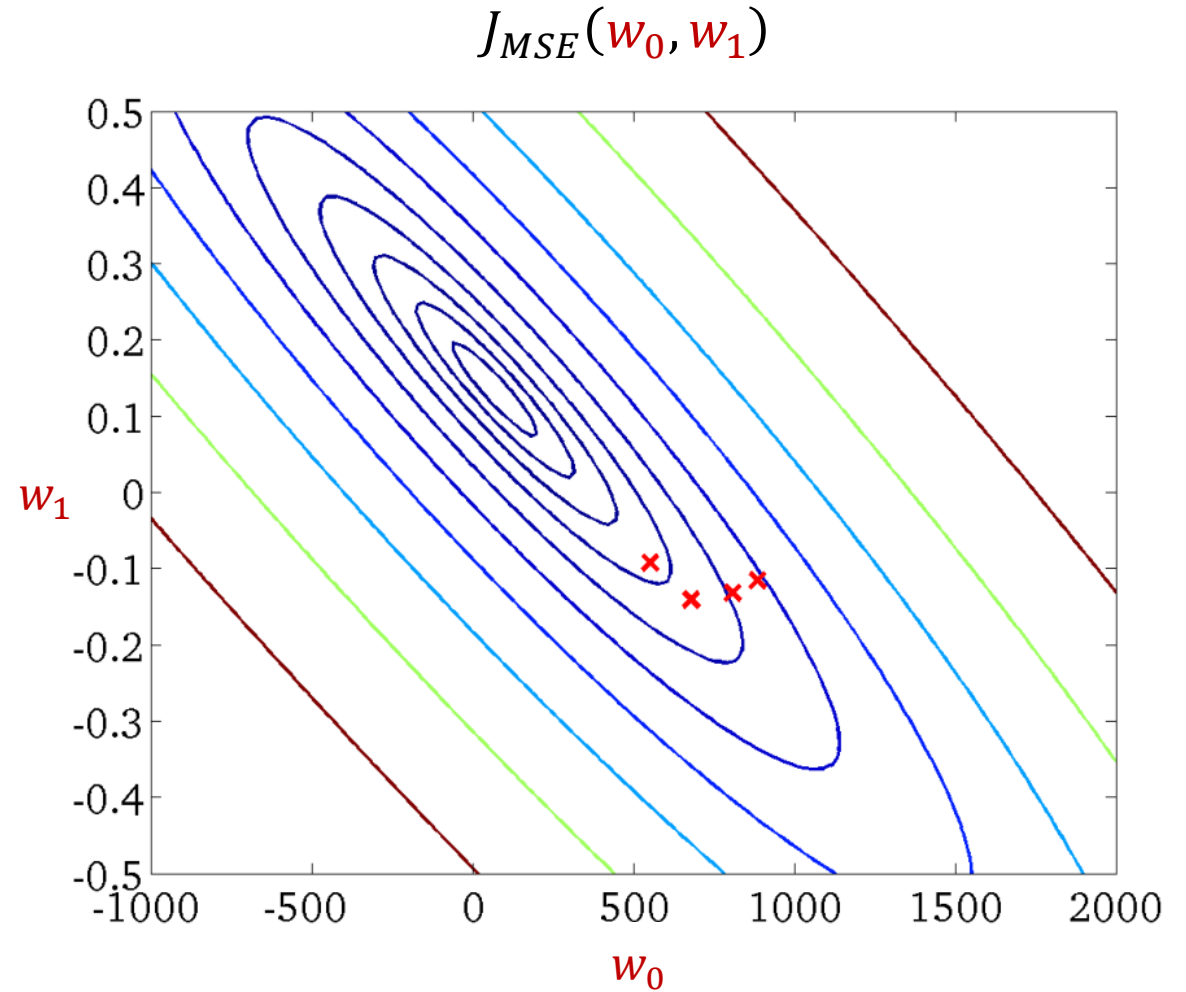
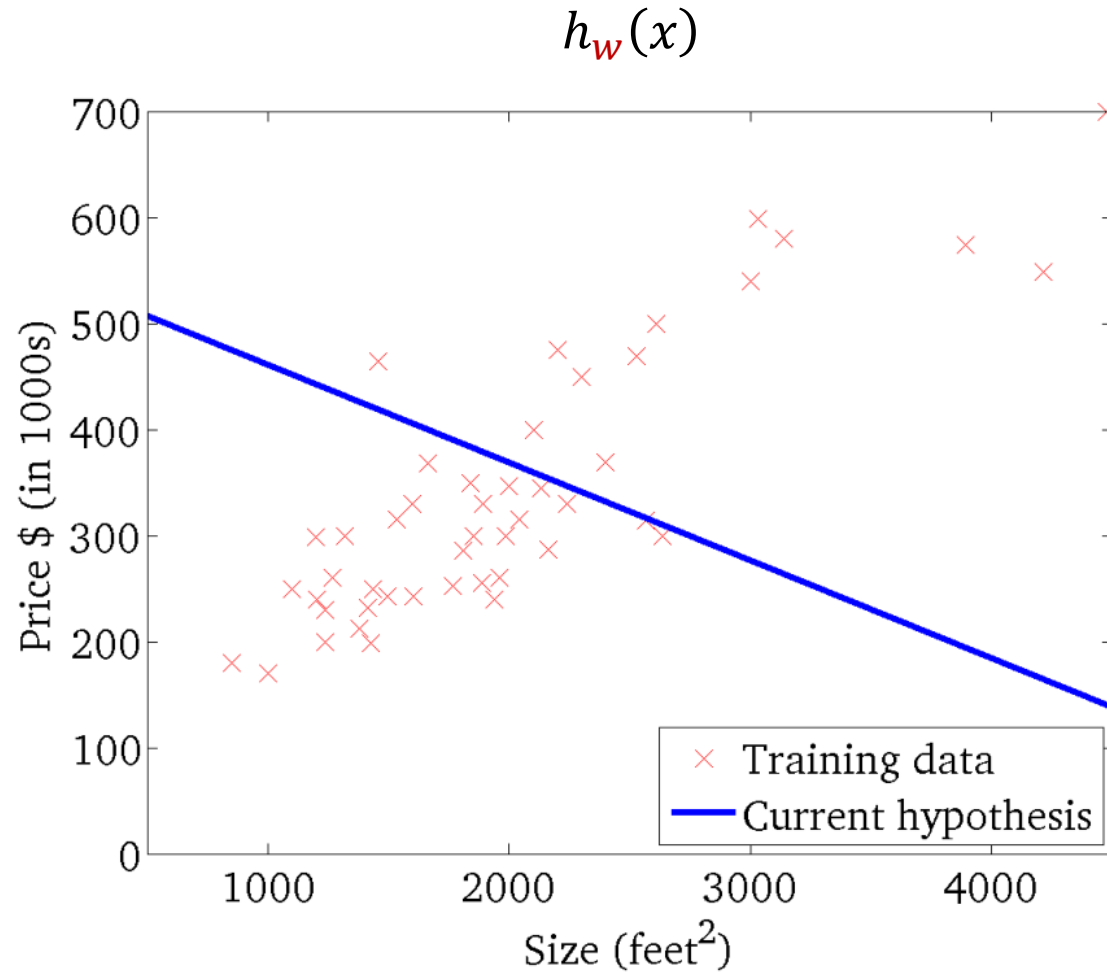
Linear Regression with Gradient Descent



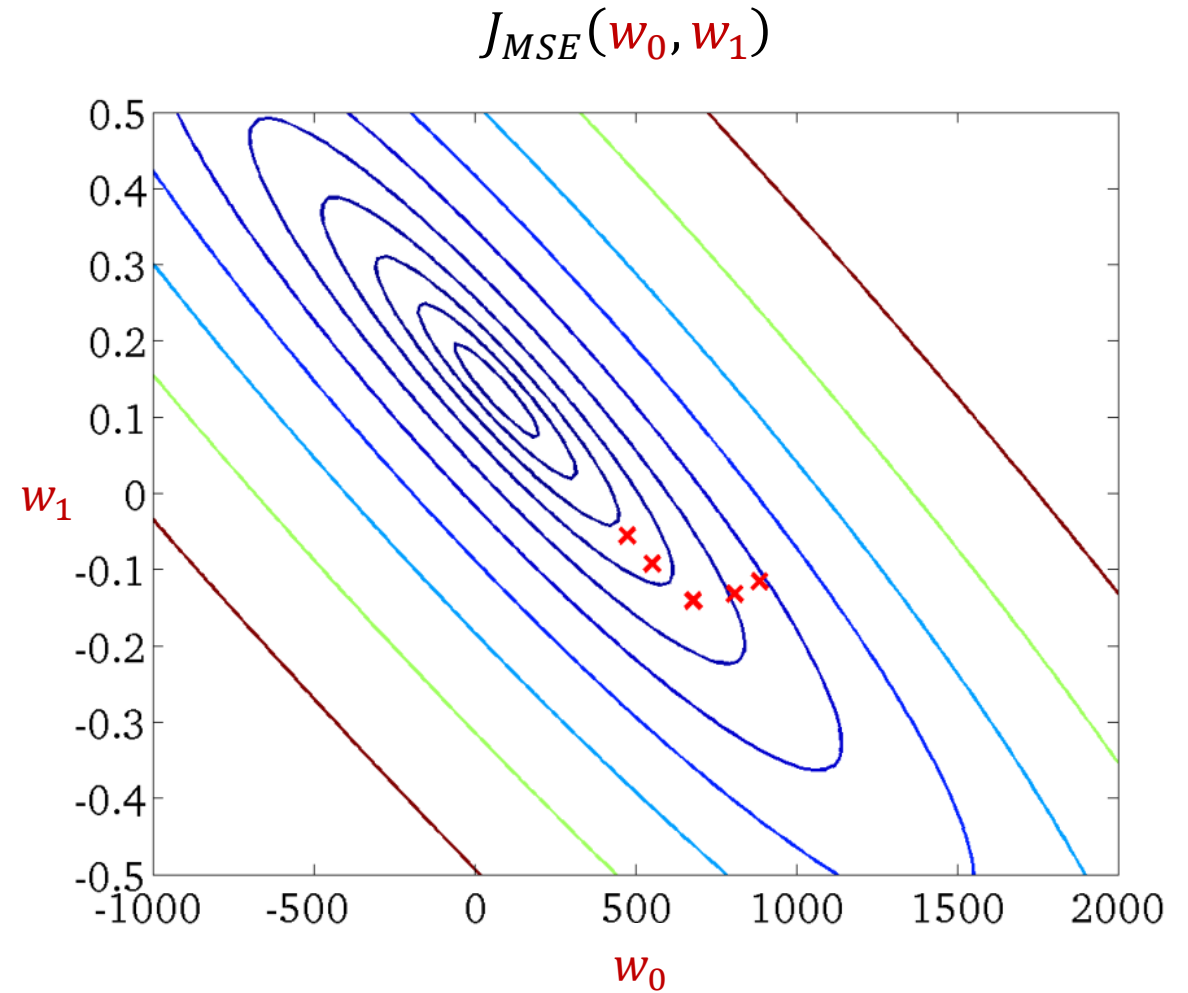
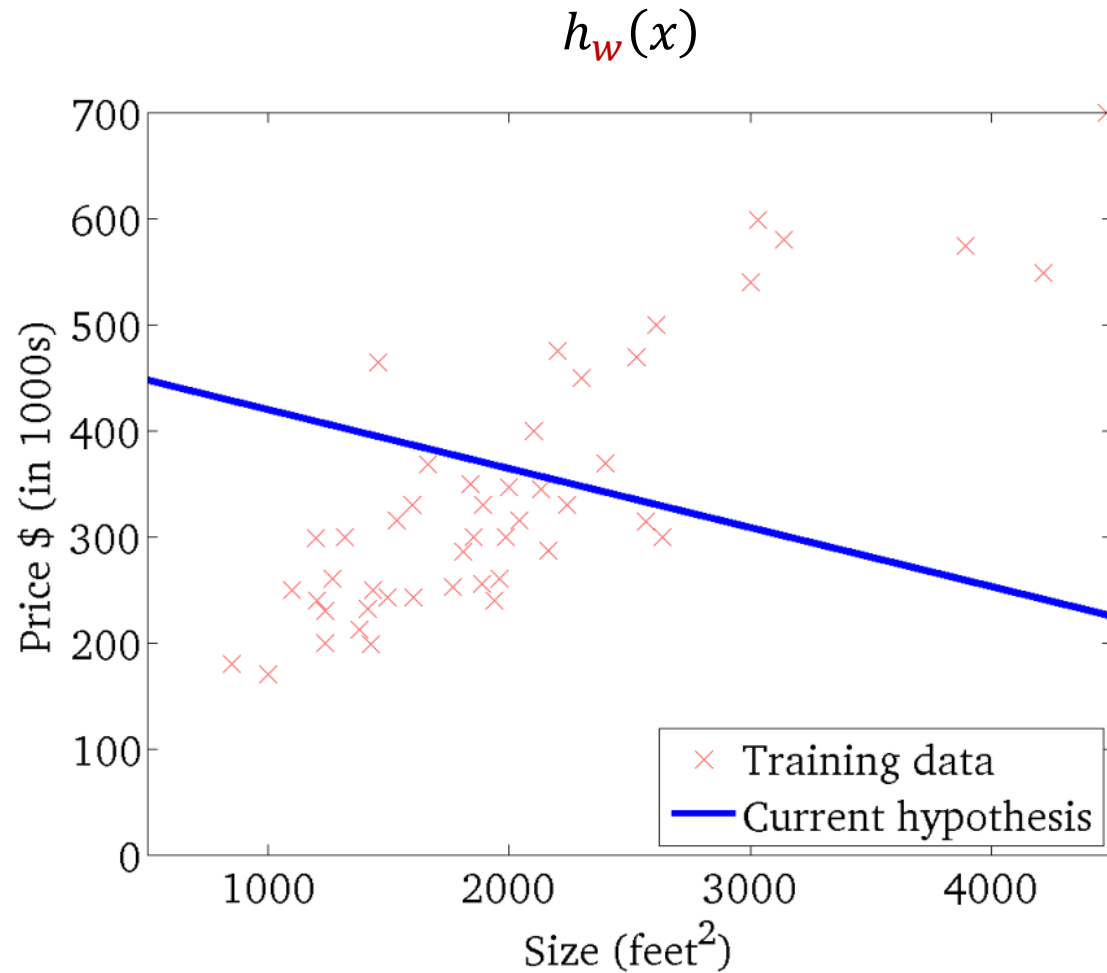
Linear Regression with Gradient Descent



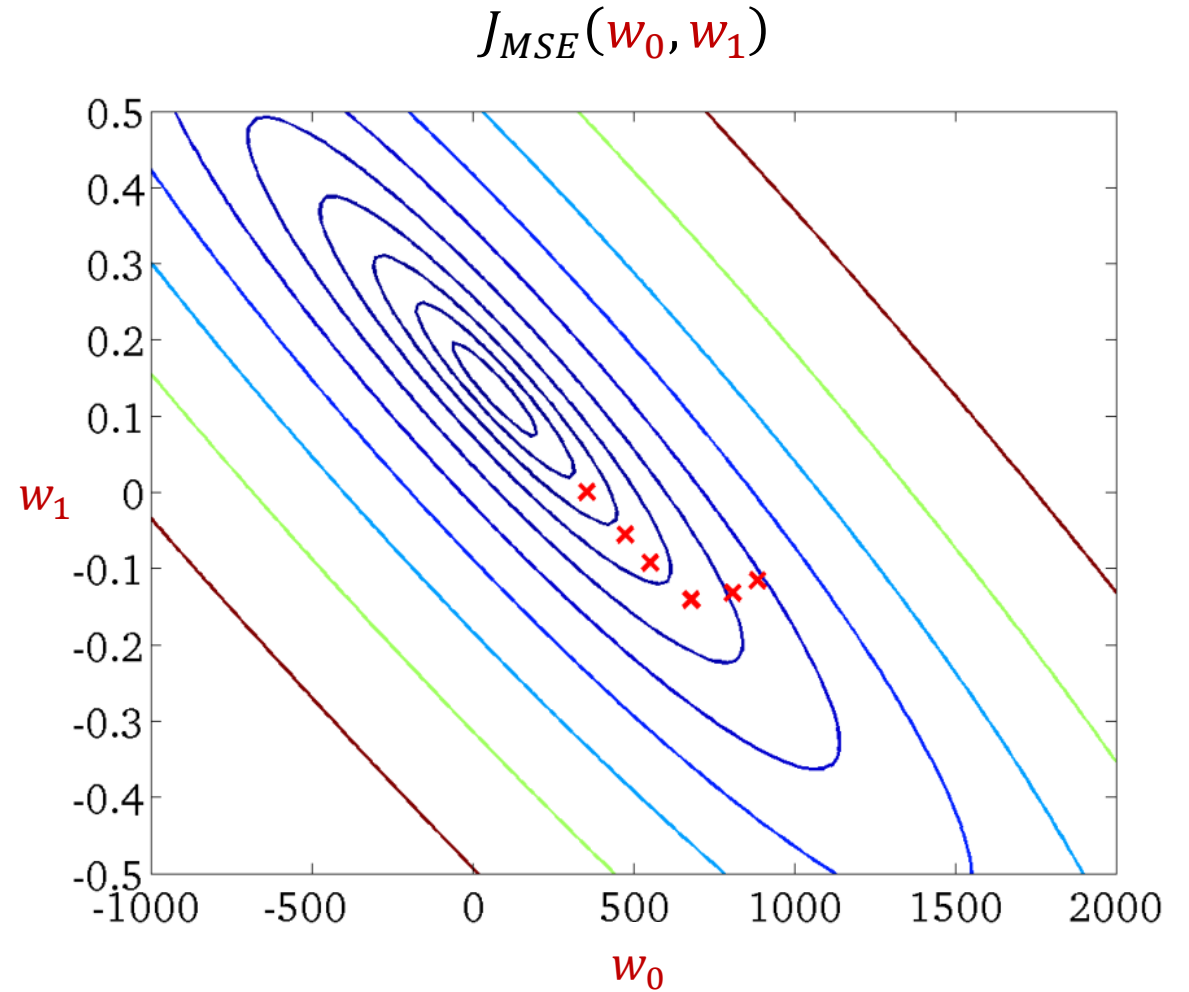
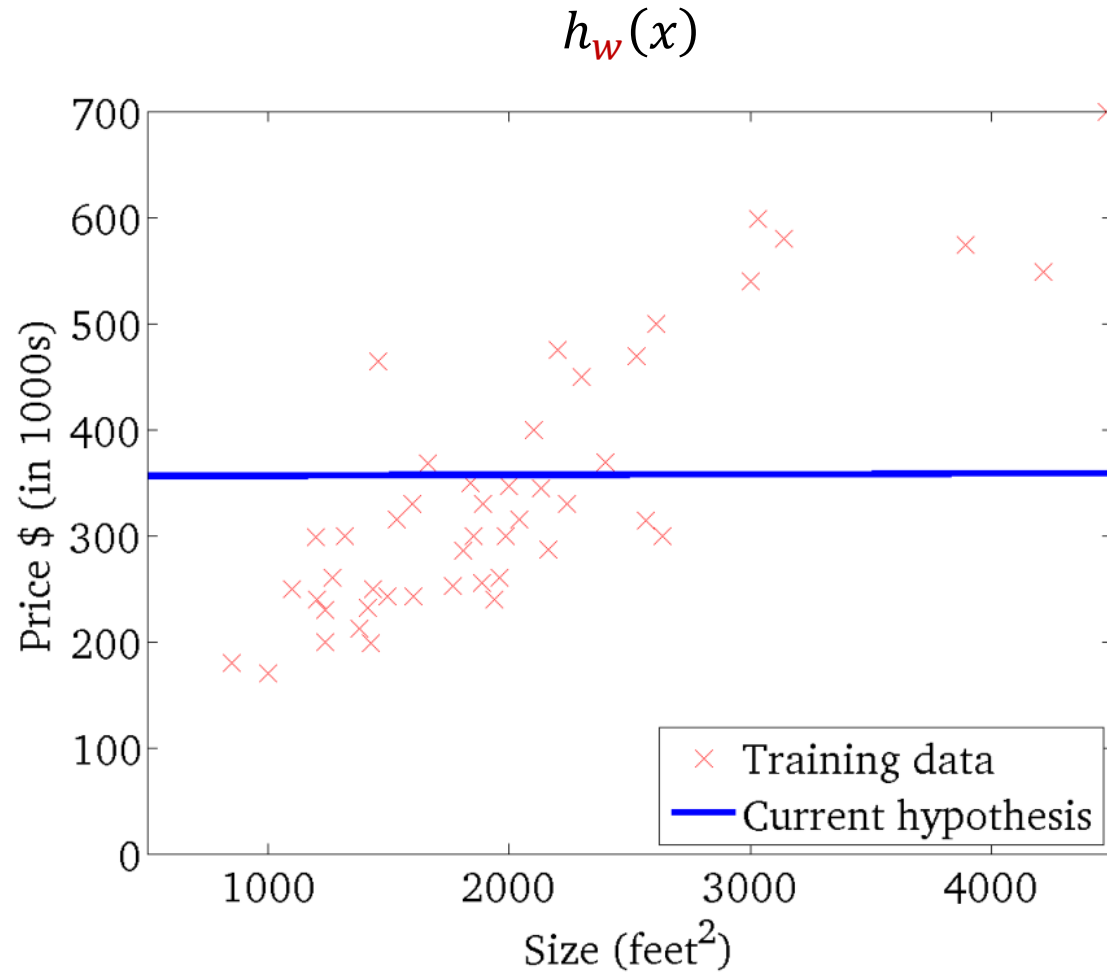
Linear Regression with Gradient Descent



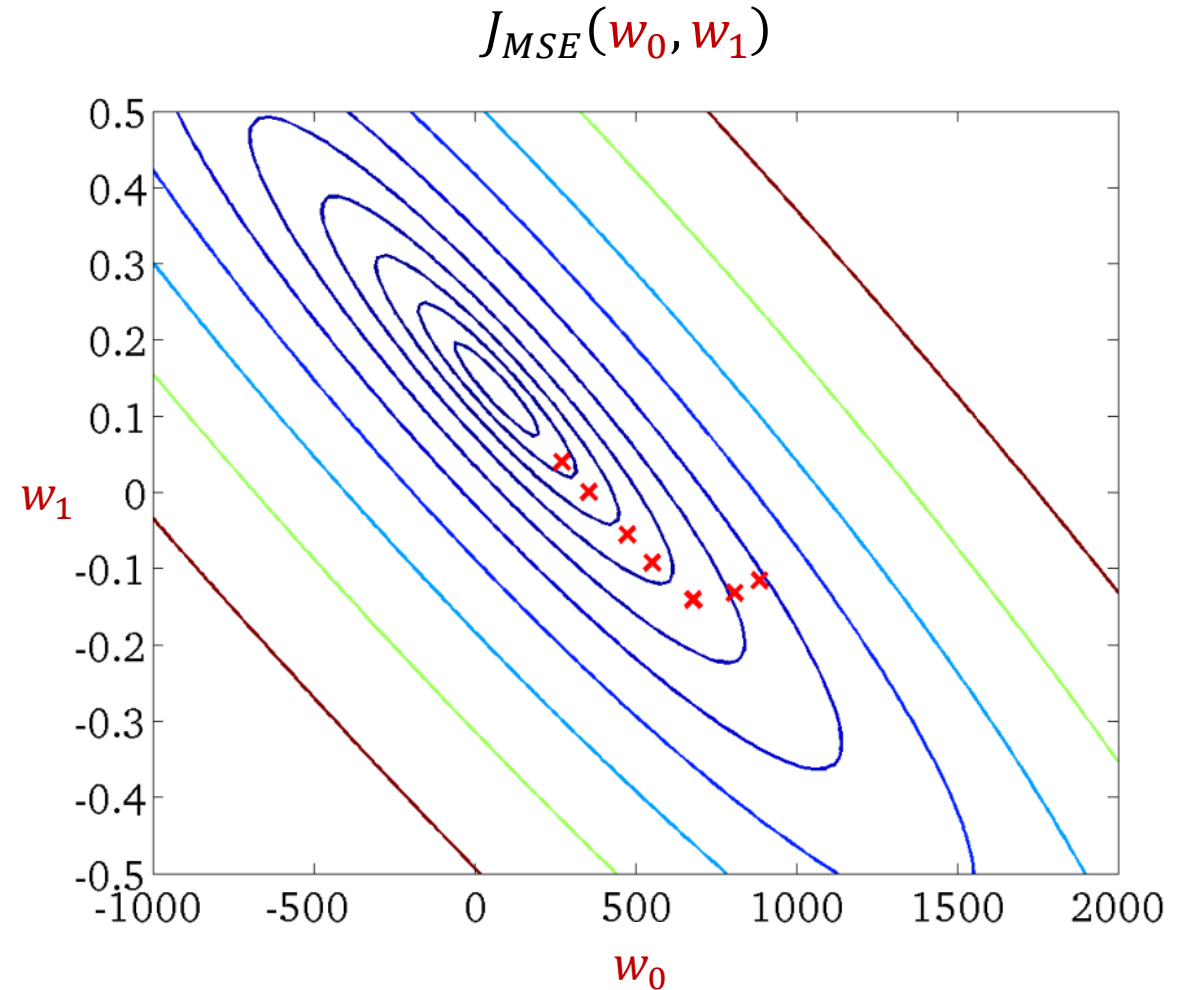
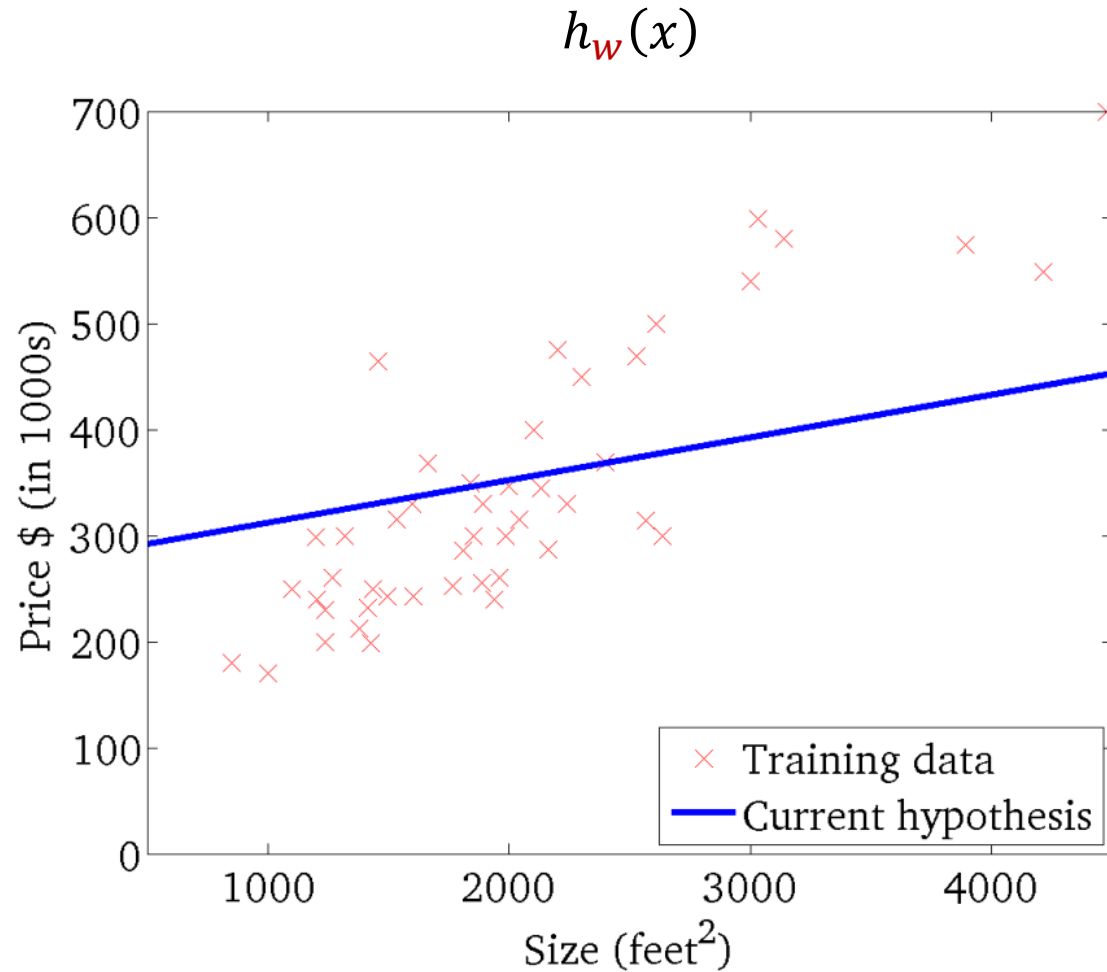
Linear Regression with Gradient Descent



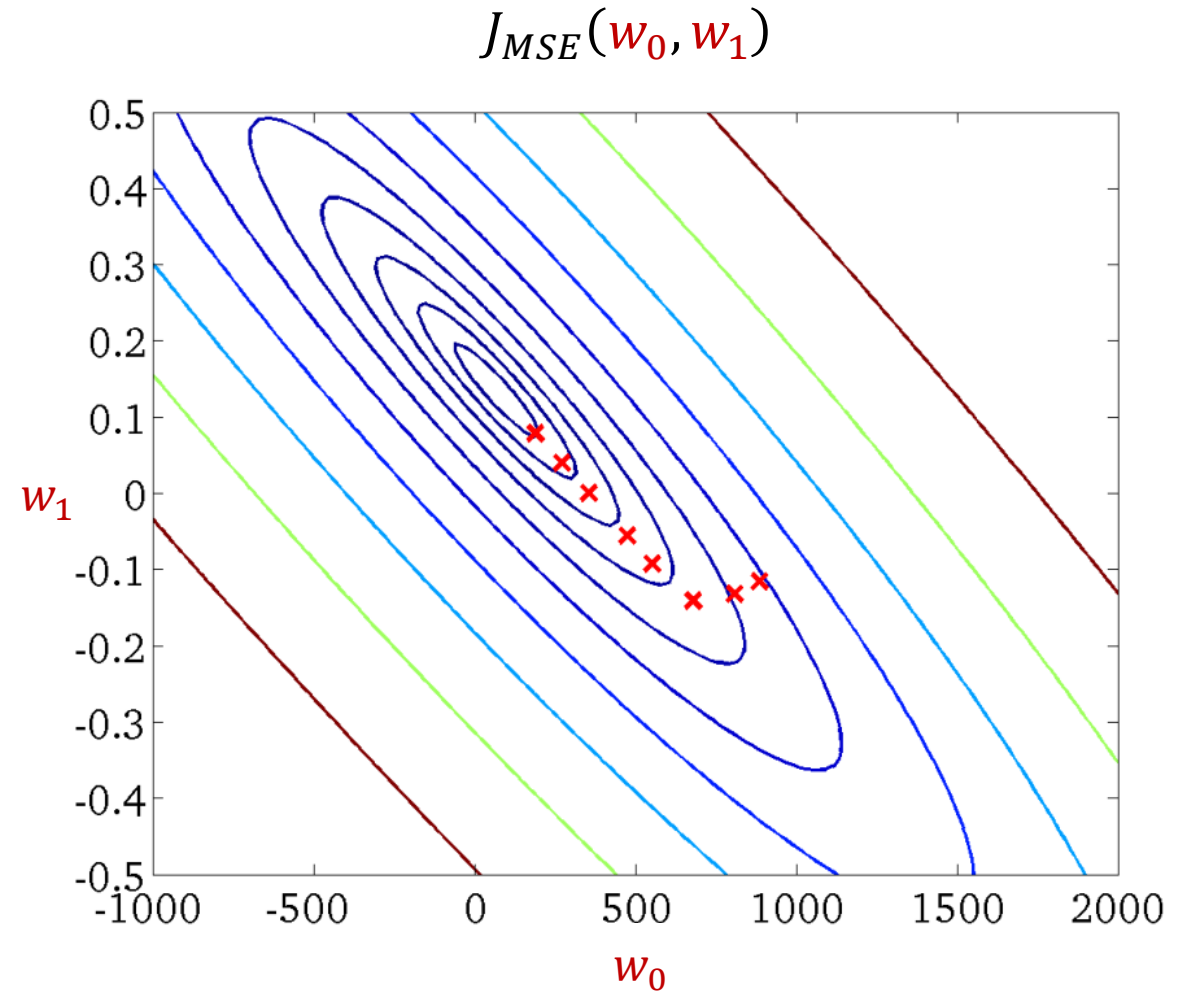
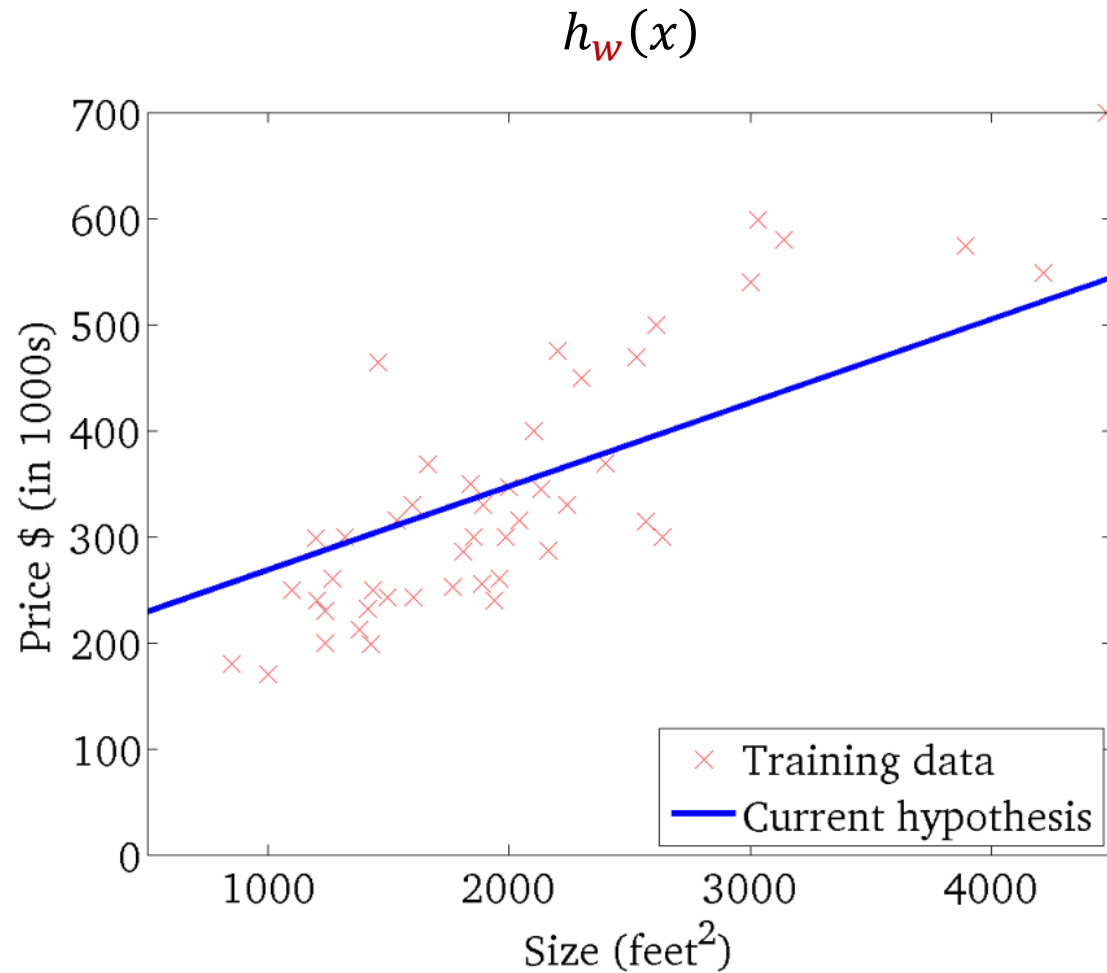
Linear Regression with Gradient Descent



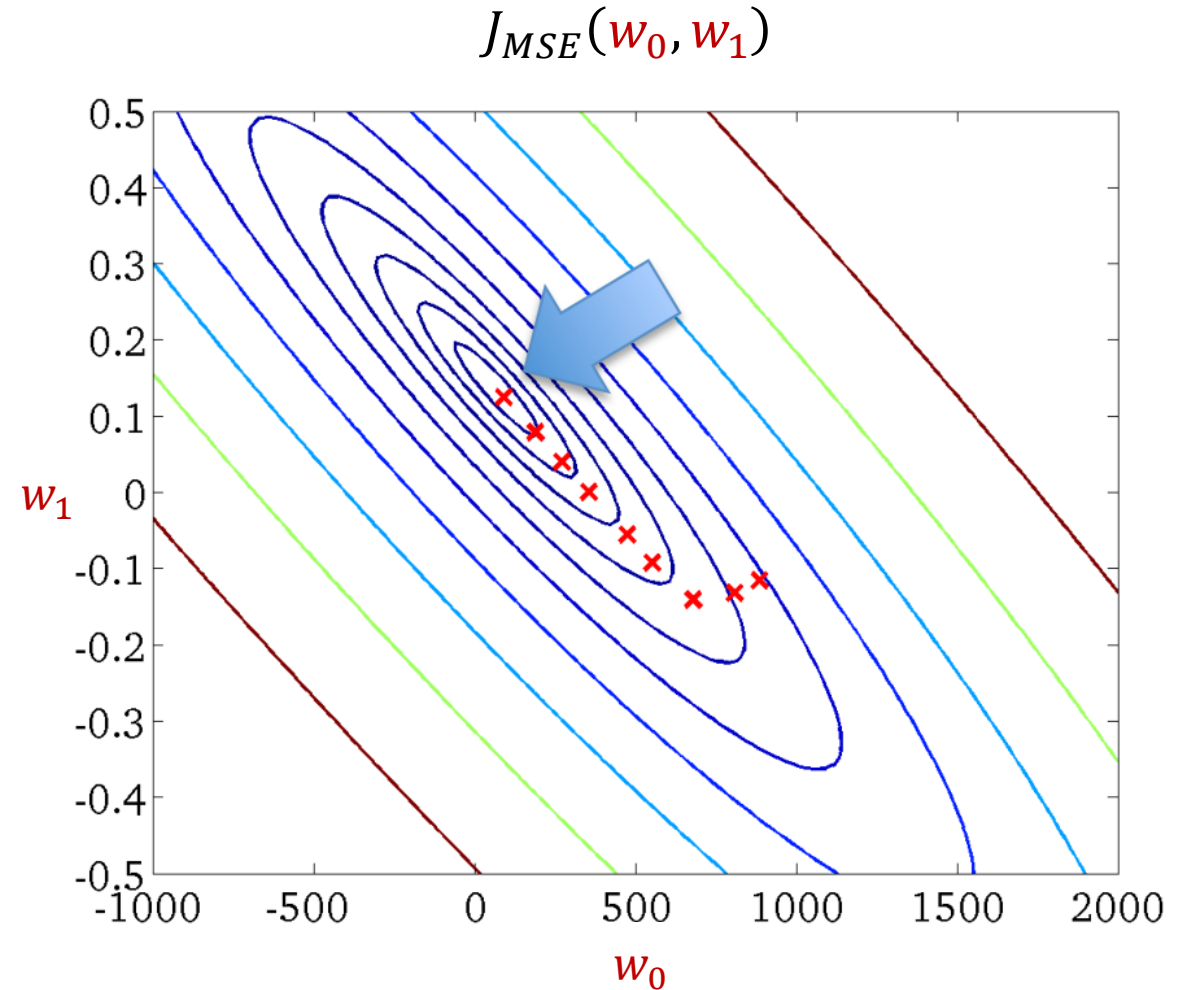
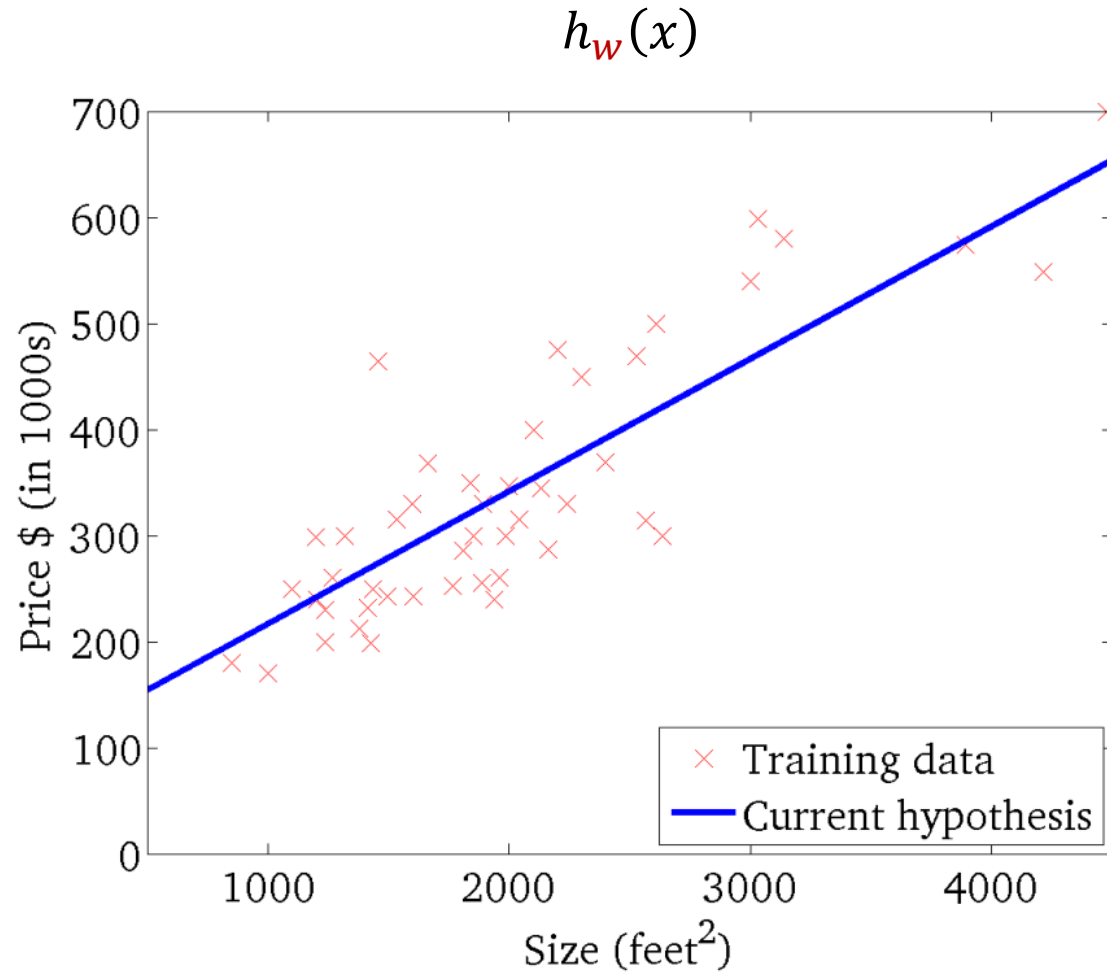
Linear Regression with Gradient Descent



Linear Regression with Gradient Descent



Linear Regression with Gradient Descent



Background: Convexity

Definition: Consider a real-valued one-dimensional function and any line segment connecting any **two distinct points** on the function's graph.

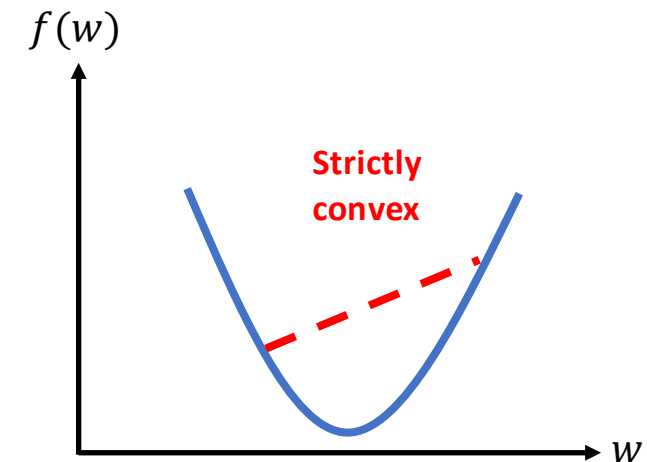
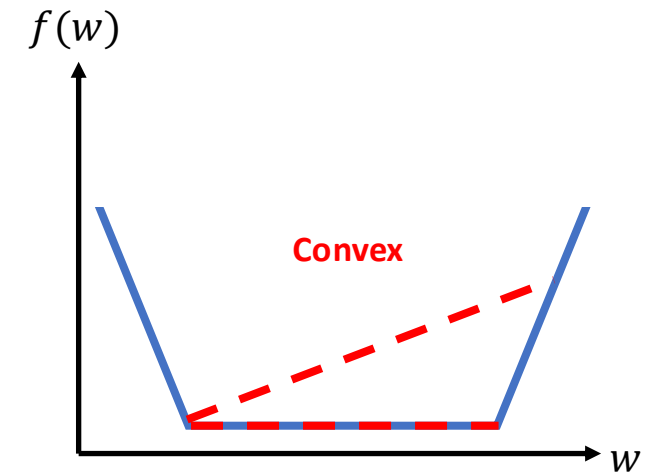
The function is called:

- **Convex** if the line lies above or on the graph between the two points.
- **Strictly convex** if the line always lies above the graph between the two points.

For a two-dimensional function, imagine a bowl-shaped landscape.

Theorem: For a convex function, any local minimum is also a global minimum.

Theorem: A strictly convex function has at most one unique global minimum.



Linear Regression – Analysis

Theorem: MSE loss function for a linear regression model is a convex function with respect to the model's parameters.

Definition: Linearly dependent features. A feature is linearly dependent on other features, if it is a linear combination of the other features, e.g., $x_3 = c_1x_1 + c_2x_2$ for some constants c_1, c_2 .

Theorem: If the features in the training data are **linearly independent**, the function (MSE + linear model) is strictly convex.

Theorem: For a **linear regression model** with a **Mean Squared Error (MSE)** loss function, the **Gradient Descent** algorithm is guaranteed to converge to a **global minimum**, provided the learning rate is chosen appropriately. If the features are linearly independent, this global minimum is unique.

A Problem with Gradient Descent

Features may have different scales

Suppose that you have two samples (given on the right) and the following loss function.

$$\begin{aligned} J(\mathbf{w}) &= (\mathbf{w}_1 x_1^{(1)} + \mathbf{w}_2 x_2^{(1)} - y^{(1)})^2 + (\mathbf{w}_1 x_1^{(2)} + \mathbf{w}_2 x_2^{(2)} - y^{(2)})^2 \\ &= (1\mathbf{w}_1)^2 + (10\mathbf{w}_2)^2 \end{aligned}$$

# Guest Rooms (0 - 3)	Distance to landfills in km (0 – 30 km)	Target: Price Premium (\$K)
1	0	0
0	10	0

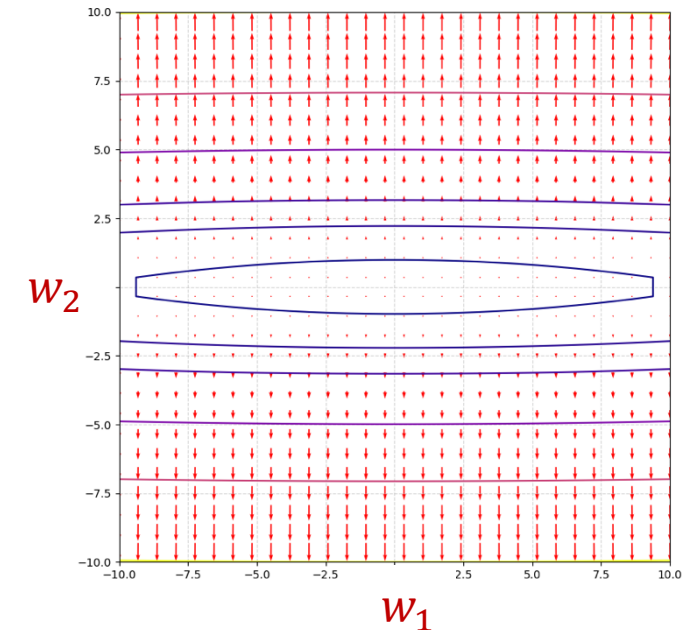
The gradient is

$$\nabla J(\mathbf{w}) = \begin{bmatrix} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_1} \\ \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_2} \end{bmatrix} = \begin{bmatrix} 2\mathbf{w}_1 \\ 200\mathbf{w}_2 \end{bmatrix}$$

Gradient descent update:

$$\begin{aligned} \mathbf{w}_1 &\leftarrow \mathbf{w}_1 - \gamma 2\mathbf{w}_1 \\ \mathbf{w}_2 &\leftarrow \mathbf{w}_2 - \gamma 200\mathbf{w}_2 \end{aligned}$$

Gradient of $J(\mathbf{w})$ for this example



While this example is contrived, this effect shows up often in loss function minimizations.

Solutions for Features with Different Scales

Scale features to be within a **common** range. Also known as **normalization**.

Min-max scaling

$$x'_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

$\min(x_i)$ and $\max(x_i)$ denote the minimum and the maximum value of a feature x_i in dataset D

Scales the features to be within $[0,1]$.

It is also common to scale the features to be within $[-1,1]$

Standardization

$$x'_i = \frac{x_i - \mu_i}{\sigma_i}$$

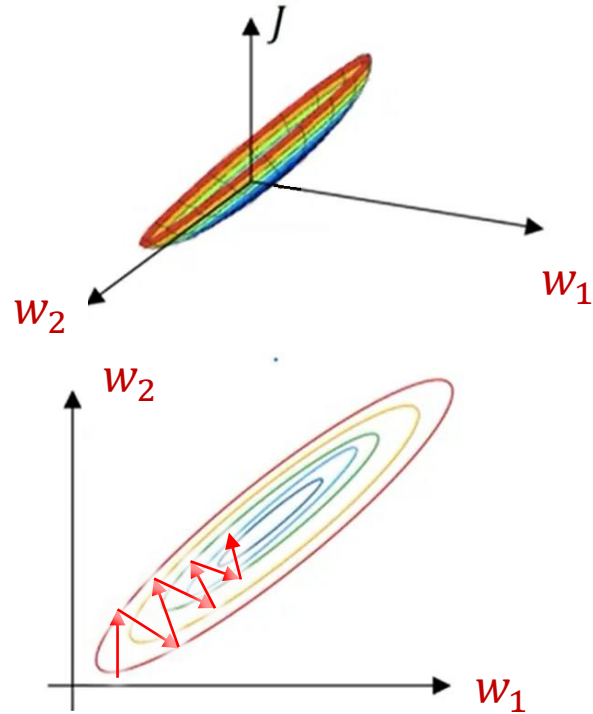
μ_i and σ_i is the mean and the standard deviation of feature x_i in dataset D

This method transforms features to have a mean of 0 and a standard deviation of 1

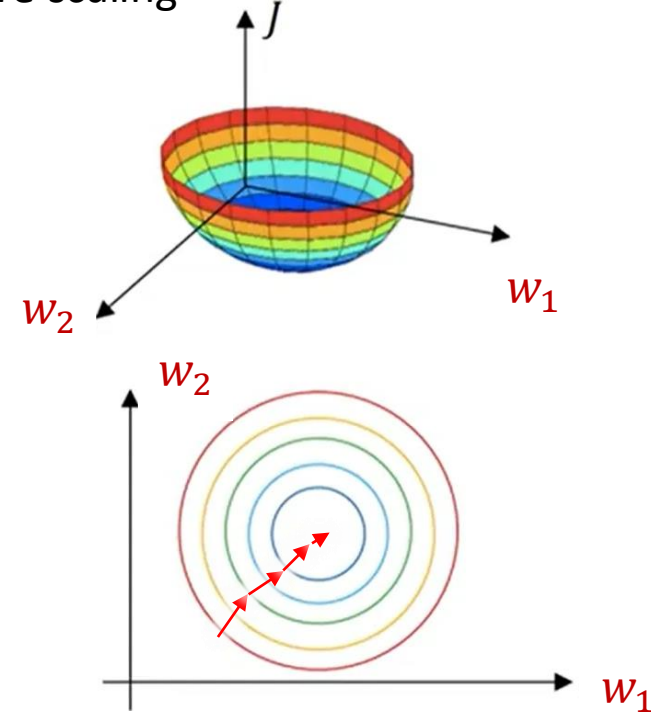
Other solution: Different learning rate γ_j for each weight.

Solutions for Features with Different Scales

Without feature scaling



With feature scaling

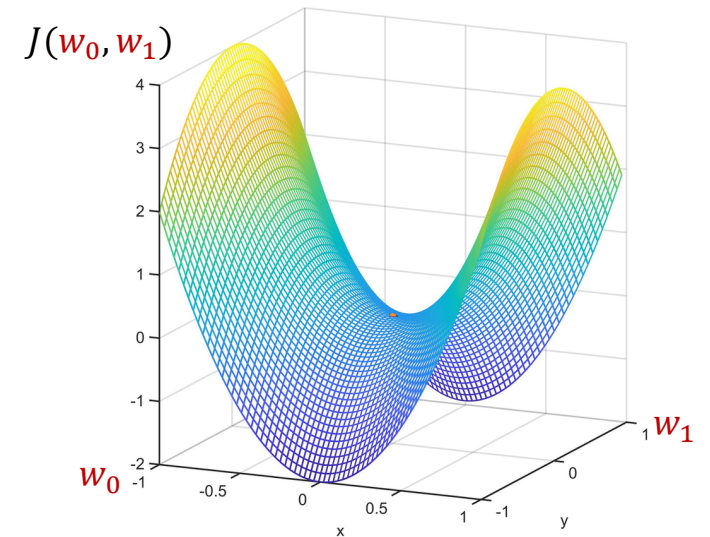


Other Problems with Gradient Descent

- On large datasets, gradient descent will be **very slow** because it needs to consider the entire dataset to compute the gradient.
- Gradient descent may get stuck at a **local minimum** or **plateau** on non-convex optimization, which we will encounter in future lectures

$$\frac{\partial J_{MSE}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^T x^{(i)} - y^{(i)}) x_j^{(i)}$$

$w_j \leftarrow w_j - \gamma \frac{\partial J_{MSE}(w_0, w_1, \dots)}{\partial w_j}$

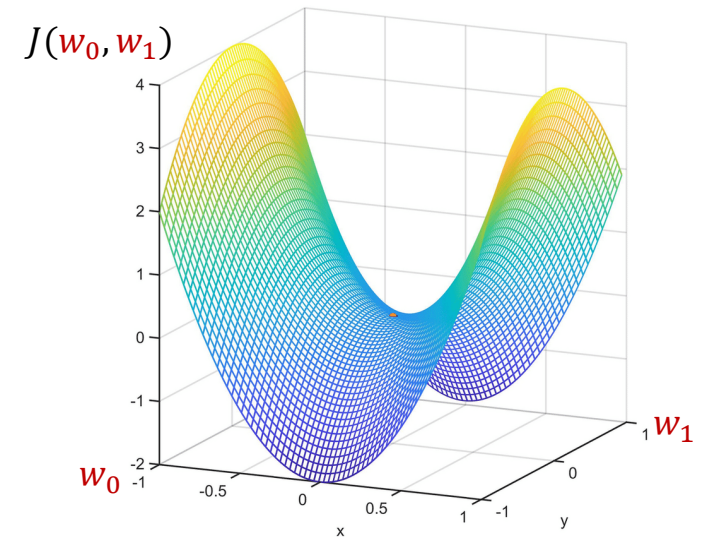


Variants of Gradient Descent

- (Batch) Gradient Descent
 - Consider all training examples
- **Mini-batch** Gradient Descent
 - Consider a subset of training examples at a time
 - Cheaper (Faster) per iteration
 - Noisy gradient, may help escape local minima/plateau
- **Stochastic** Gradient Descent (SGD)
 - Select one random training example at a time
 - Cheapest (Fastest) per iteration
 - More noisy gradient, may help escape local minima /plateau

$$\frac{\partial J_{MSE}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (w^T x^{(i)} - y^{(i)}) x_j^{(i)}$$


$$w_j \leftarrow w_j - \gamma \frac{\partial J_{MSE}(w_0, w_1, \dots)}{\partial w_j}$$



Learning Algorithms: Comparison

$$w_j \leftarrow w_j - \gamma \frac{\partial J(w_0, w_1, \dots)}{\partial w_j}$$

$$w = (X^T X)^{-1} X^T Y$$

	Gradient Descent	Normal Equation
Need to choose learning rate γ	Yes	No ✓
Iteration(s)	Many	None, closed-form solution ✓
Large number of features d ?	No problem ✓	Slow, $(X^T X)^{-1} \rightarrow O(d^3)$
Feature scaling?	May be necessary	Not necessary ✓
Constraints	- ✓	$X^T X$ needs to be invertible

Outline

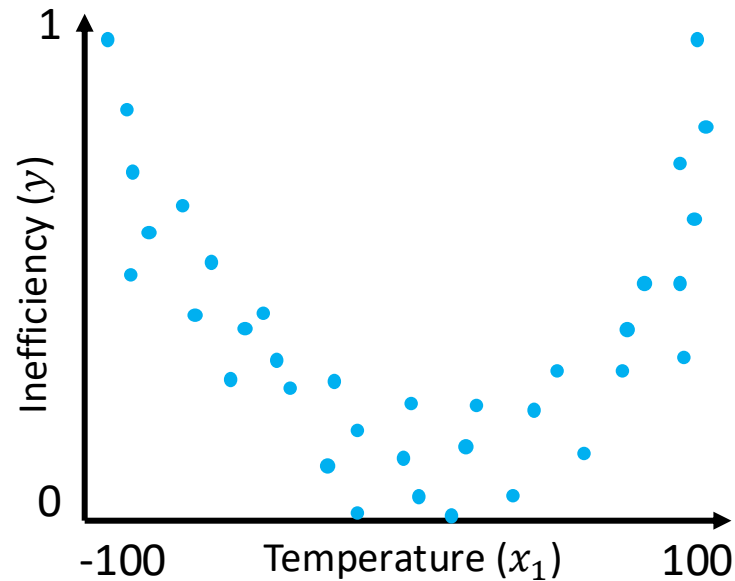
- Linear Regression
 - Data
 - Model
 - Loss
- Learning via Normal Equations
- Learning via Gradient Descent
 - Gradient Descent Algorithm
 - Variants: Mini-batch, stochastic
- **Feature Transformation**

Example: Engine Inefficiency Prediction

Given an engine temperature, what is its inefficiency? (lower means more efficient)

i	$x_1^{(i)}$	$y^{(i)}$
1	-100	1
2	-50	0.25
3	0	0.05
4	50	0.27
5	100	0.98
...	...	...

Dataset D



$$h_w(x) = w_0 x_0 + w_1 x_1$$

$x_0 = 1$ (dummy variable)

How can we use linear models here?

Feature Transformation

We can take a d -dimensional feature vector and transform it into a M -dimensional feature vector. Usually $M \geq d$.

$$x \in \mathbb{R}^d \quad \rightarrow \quad \phi(x) \in \mathbb{R}^M$$

Here,

- The function ϕ is called a **feature transformation** or **feature map**,
- The vector $\phi(x)$ is called **transformed features**.

Feature Transformation – Example

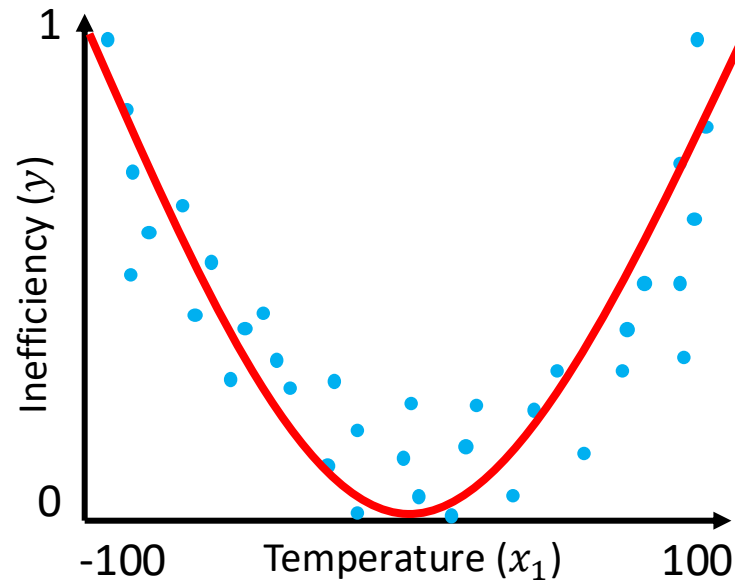
- **Linear:** $\phi(x) \rightarrow x$.
- **Polynomial** features of degree k , $\phi_{pk}(x)$.
 - Polynomial **combinations** of the features with degree $\leq k$.
 - When used with linear regression, this is called **polynomial regression**.
 - Example of 2-dimensional feature with degree-2 polynomial transformation:
$$\phi_{p2}([x_1 \ x_2]) \rightarrow [x_1 \ x_2 \ x_1x_2 \ x_1^2 \ x_2^2]$$
- **Log:** For each x_i , transform $x_i \rightarrow x_i$ and $\log(x_i)$.
 - Example: $\phi_{log}([x_1 \ x_2]) \rightarrow [x_1 \ x_2 \ \log(x_1) \ \log(x_2)]$
- **Exponential:** For each x_i , transform $x_i \rightarrow x_i$ and $\exp(x_i)$.
 - Example: $\phi_{exp}([x_1 \ x_2]) \rightarrow [x_1 \ x_2 \ \exp(x_1) \ \exp(x_2)]$

Linear Model with Feature Transformation

Given an engine temperature, what is its inefficiency (lower means more efficient)?

i	$x_1^{(i)}$	$y^{(i)}$
1	-100	1
2	-50	0.25
3	0	0.05
4	50	0.27
5	100	0.98
...	...	...

Dataset D



x_1	-100	-50	0	50	100
x_2	10,000	2,500	0	2,500	10,000

$$h_w(x) = w_0 x_0 + w_1 x_1 + w_2 x_2$$

$x_0 = 1$ (dummy variable)

Use **polynomial feature** transformation degree 2:

$$\phi_{p2}([x_1]^T) \rightarrow [x_1 \ x_1^2]^T$$

Linear model:

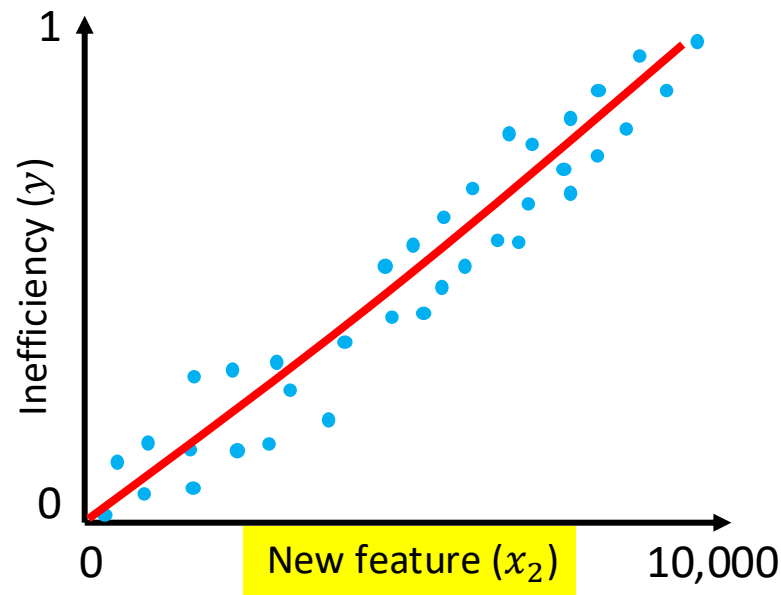
$$h_w(\phi_{p2}(x)) = 0 + 0x_1 + \frac{1}{10,000}x_2$$

Linear Model with Feature Transformation

Given an engine temperature, what is its inefficiency (lower means more efficient)?

i	$x_2^{(i)}$	$y^{(i)}$
1	10,000	1
2	2,500	0.25
3	0	0.05
4	2,500	0.27
5	10,000	0.98
...	...	...

Dataset D



x_1	-100	-50	0	50	100
x_2	10,000	2,500	0	2,500	10,000

$$h_w(x) = w_0 x_0 + w_1 x_1 + w_1 x_2$$

$x_0 = 1$ (dummy variable)

Use **polynomial feature** transformation degree 2:

$$\phi_{p2}([x_1]^T) \rightarrow [x_1 \ x_1^2]^T$$

Linear model:

$$h_w(\phi_{p2}(x)) = 0 + 0x_1 + \frac{1}{10,000}x_2$$

Data is linear from the perspective of feature x_2

LM with Feature Transformation – Analysis

Theorem: The convexity of linear regression with MSE is preserved **with feature transformation**.

A way to think about it:

- Feature transformation can be seen as **creating a new dataset** with a **new set of features**.
- The model remains a linear model but operates on the **new set of features**.

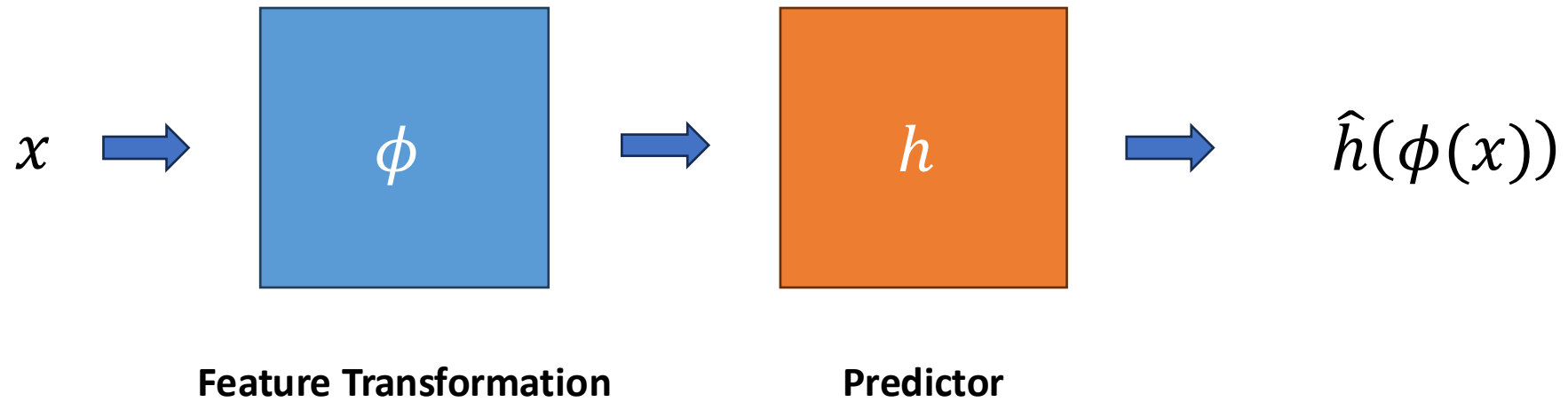
A General View of ML Models

A machine learning model $\hat{h}$ is usually of the following form

$$\hat{h}(x) = h(\phi(x))$$

- ϕ is called the **feature transformation/mapping/extraction** function.
 - It may do nothing, i.e., use a linear transformation $\phi(x) = x$
 - It may be handcrafted, e.g., polynomial features
 - It may be learned, e.g., neural networks, which we will introduce in future lectures.
- h is called the **predictor**.
 - It is usually (but not always) a simple (parameterized) model, e.g., a linear model.
 - It is the one performing prediction, i.e., regression, classification.

A General View of ML Models



Summary

- Linear Regression: **fitting a line** to data
- Linear Model
 - d dimensional input features: $h_{\mathbf{w}}(x) = \sum_{j=0}^d \mathbf{w}_j x_j = \mathbf{w}^T x$
- Finding the best function, i.e., one that minimizes the loss
 - Normal Equation: **set derivative to 0, solve**
 - Gradient Descent
 - Gradient Descent Algorithm: **follow –gradient** to reduce error
 - Linear Regression with Gradient Descent: **convex** optimization, **one minimum**
 - Problem: Features of Different Scales: **normalize!**
 - Variants of Gradient Descent: batch, mini-batch, stochastic
- Feature transformation:
 - Linear, polynomial, log, exp features
 - Fit complex function, still convex with MSE

Further Reading (Optional)

- History of regression (The origins and uses of regression analysis, 1997)
- Feature encoding, scaling (TESL, Chapter 3 and 5)
- Normal equation derivation (TESL, Section 3.2)
- Complexity of inverting matrix (INV)
- Proof convexity $\rightarrow$ global minimum (BV, Section 4.2.2)
- Proof: MSE loss function is convex for linear regression (TESL, Section 3.2)
- Different learning rate for each weight (e.g. AdaGrad)

TESL: Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.

BV: Boyd, S., & Vandenberghe, L., (2004). *Convex Optimization*. Cambridge University Press.

INV: For example, Farebrother, R.W. (1988), *Linear Least Squares Computations*.

Coming Up Next Week

- **Logistic** Regression

To Do

- **Lecture Training 5**
 - +550 EXP
 - +100 Early bird bonus

